



MineSim: A scenario-based simulation test system and benchmark for autonomous trucks in open-pit mines[☆]

Zhifa Chen^{a, *}, Guizhen Yu^a, Peng Chen^{a, *}, Guoxi Cao^a, Zheng Li^a, Yifang Zhang^a, Haoyuan Ni^a, Bin Zhou^a, Jian Sun^b, Huanyu Ban^c

^a School of Transportation Science and Engineering, Key Laboratory of Autonomous Transportation Technology for Special Vehicles, Ministry of Industry and Information Technology, Beihang University, Beijing 100191, China

^b Key Laboratory of Road and Traffic Engineering, Department of Traffic Engineering, Ministry of Education, Tongji University, Shanghai 201804, China

^c The Department of Land Surveying and Geo-Informatics, The Hong Kong Polytechnic University, Hong Kong, China

ARTICLE INFO

Keywords:

Autonomous trucks
Simulation environments
Open-pit mining
Planning tasks
Scenario-based testing
Obstacle avoidance

ABSTRACT

Simulation environments are essential for validating algorithms, evaluating system performance, and ensuring safety in autonomous driving systems before real-world deployment. Existing autonomous driving simulators are designed for urban scenarios but lack coverage of unstructured road environments in open-pit mining. This paper introduces MineSim, an open-source, scenario-based simulation test system specifically developed for planning tasks in autonomous trucks operating in open-pit mines. MineSim includes several components: automated scenario parsing, state update models for the ego vehicle, state update policies for other agents, metric evaluation, and scenario visualization tools. It incorporates numerous real-world traffic scenarios from two open-pit mines that capture the unique challenges of unstructured road environments, including irregular intersections, roads without clear lane markings, and the response lags of heavy autonomous mining trucks. Furthermore, MineSim provides scenario libraries and benchmarks for static and dynamic obstacle avoidance problems, facilitating research into planning algorithms in these complex settings. By offering reproducible testing methods and scenario data, MineSim serves as a critical resource for advancing autonomous driving technologies in non-urban and unstructured road environments (see <https://buaa-trans-mine-group.github.io/minesim>).

1. Introduction

During the development and testing of autonomous driving systems, simulation environments play a crucial role in validating algorithms, assessing system performance, and ensuring safety before real-world deployment. These environments allow for rigorous testing under controlled conditions, enabling developers to evaluate how autonomous vehicles respond to a wide range of driving scenarios, including those that are difficult or dangerous to replicate in the real world. Currently, the majority of research focuses on urban driving scenarios. Autonomous driving simulators such as LGSVL (Rong et al., 2020), AirSim (Shah et al., 2018), and CARLA (Dosovitskiy et al., 2017) provide comprehensive testing capabilities for autonomous vehicles, ranging from basic vehicle dynamics to complex urban traffic interactions. These

simulators also support the simulation of sensor data (e.g., cameras, LiDAR, radar), vehicle behavior modeling, and high-fidelity 3D environment rendering. Additionally, some testing systems specialize in planning tasks, such as CommonRoad (Althoff et al., 2017), NuPlan (Caesar et al., 2022), ScenarioNet (Li et al., 2023b), and Waymax (Gulino et al., 2023), offering scenario-based simulation frameworks for testing planning algorithms. However, these simulators are primarily designed for urban environments and do not address the needs of open-pit mining environments.

As an industrial setting where autonomous driving technology is increasingly applied, open-pit mines lack an open-source, scenario-based automated simulation tool. A few tools have recently emerged to address autonomous driving in open-pit mining, such as OpenMines (Meng et al., 2024), which provides an open-source framework for

[☆] This article is part of a special issue entitled: 'Safety validation' published in Accident Analysis and Prevention.

^{*} Corresponding author.

E-mail addresses: chenzhifa@buaa.edu.cn (Z. Chen), yugz@buaa.edu.cn (G. Yu), cpeng@buaa.edu.cn (P. Chen), caoguxi@buaa.edu.cn (G. Cao), zy2313307@buaa.edu.cn (Z. Li), zf2213129@buaa.edu.cn (Y. Zhang), nihyu@buaa.edu.cn (H. Ni), binzhou@buaa.edu.cn (B. Zhou), sunjian@tongji.edu.cn (J. Sun), huanyu.ban@connect.polyu.hk (H. Ban).

<https://doi.org/10.1016/j.aap.2025.107938>

Received 14 October 2024; Received in revised form 3 January 2025; Accepted 21 January 2025

Available online 8 February 2025

0001-4575/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

modeling mining systems, developing scheduling algorithms for mining truck fleets, and assessing their performance. AutoMine (Li et al., 2022b) is the first open-source dataset focused on perception tasks in open-pit mining. However, there is still a lack of scenario data and closed-loop simulation tools for testing obstacle avoidance planning tasks in open-pit mines on-road scenarios. One of the few works that addresses this gap is refer to Li et al. (2023a), which abstracted loading and unloading operations in open-pit mines into parking tasks in unstructured road environments. In 2023, they also organized a trajectory planning competition for automated parking (TPCAP), providing reproducible benchmark algorithms and unstructured road scenarios.

Given the unique characteristics of open-pit mines with unstructured road environments, including (a) irregular, unsignalized intersections often with slopes; (b) roads lacking clear lane markings, with irregular boundaries and uneven widths; and (c) large, heavy mining trucks with significant inertia, there is a clear need for tools that offer reproducible testing environments and scenario data for planning tasks in these settings. To address this, we propose MineSim, a scenario-based simulation test system designed specifically for autonomous trucks operating in open-pit mines. The main contributions of this study can be summarized as follows:

- (1) An open-source, closed-loop simulation test system: MineSim is designed for autonomous driving planning tasks in open-pit mines. It offers functionalities such as automated scenario parsing, state update models for the ego vehicle (autonomous mining truck), state update policies for other agents, metric evaluation component, and scenario visualization tools. MineSim incorporates numerous real-world traffic scenarios from two open-pit mines (see <https://buaa-trans-mine-group.github.io/minesim>).
- (2) A scenario library and benchmark for dynamic obstacle avoidance: This scenario library and benchmark is specifically designed to facilitate research on planning algorithms in conflict scenarios involving other vehicles at irregular intersections within open-pit mines.
- (3) A scenario library and benchmark for static obstacle avoidance: This scenario library and benchmark focuses on planning algorithms designed to handle static obstacles frequently encountered by large mining trucks operating in unstructured road environments.

These contributions offer reproducible testing tools and benchmarks for autonomous driving planning tasks in open-pit mines, establishing MineSim as a critical resource for advancing research in non-urban and unstructured road environments.

2. Related work

2.1. Simulators for autonomous driving

Due to traffic safety concerns, it is challenging to conduct large-scale training and testing in the real world. Therefore, simulators are essential tools in developing autonomous driving systems, allowing for the testing and validation of various components such as perception, decision-making, and control in a virtual environment. These simulators can be divided into general-purpose and specialized tools, each addressing different aspects of autonomous driving.

General-purpose simulators such as LGSVL (Rong et al., 2020), AirSim (Shah et al., 2018), and CARLA (Dosovitskiy et al., 2017) provide flexibility and scalability for simulating diverse driving environments. As open-source platforms, they offer comprehensive capabilities for testing various aspects of autonomous driving, ranging from essential vehicle dynamics to complex urban traffic interactions. These simulators are commonly used for tasks such as sensor data simulation (e.g., cameras, LiDAR, radar), vehicle behavior modeling, and high-fidelity 3D

environment rendering. LGSVL (Rong et al., 2020) supports high-fidelity simulations of urban and highway environments, making it ideal for scenarios like city driving, highway merging, and traffic intersection handling. Its ability to simulate multi-sensor data, including LiDAR, radar, and cameras, makes it versatile for evaluating perception and decision-making systems. AirSim (Shah et al., 2018), developed by Microsoft, supports aerial and ground-based vehicles, including autonomous cars. Built on the Unreal Engine, it provides high-quality 3D simulations with realistic physics and sensor modeling for cameras, LiDAR, and IMUs. AirSim's emphasis on reinforcement learning and AI-driven research, combined with its integration with Python and C++, makes it widely used in academic and research environments. However, its limitations include the lack of pre-built driving scenarios, which require users to design custom environments, and the high hardware demands of the Unreal Engine, which can limit scalability. CARLA (Dosovitskiy et al., 2017), designed specifically for urban driving scenarios, is widely adopted in academic research and industrial development. It offers detailed traffic simulation, realistic environment rendering, and extensive support for training and testing machine learning models in autonomous driving. However, its high computational requirements for high-fidelity simulations can be a limiting factor in large-scale testing.

Specialized simulators have been developed to address specific challenges in autonomous driving, such as advanced driver assistance systems (ADAS) or traffic flow modeling. For instance, PreScan (Hendriks et al., 2010) is widely used in the automotive industry for testing ADAS and autonomous vehicles, with a focus on developing and validating control algorithms. Similarly, SUMO (Lopez et al., 2018), primarily a traffic simulator, is extensively employed to study interactions between autonomous vehicles and traffic systems, enabling large-scale traffic flow simulations in complex urban environments. Some frameworks incorporate hardware such as virtual reality (VR) to evaluate the impact of autonomous vehicles on various factors, including vehicle-bicycle interactions (Xu et al., 2023), vehicle-passenger relationships (Xu et al., 2024), and emission reductions (Xu et al., 2022). Additionally, Apollo Dreamview, specifically designed for Baidu's Apollo platform (Baidu, 2024), allows developers to visualize and test autonomous driving algorithms by integrating real-world and simulated data, supporting scenarios ranging from urban driving to highways. However, its complexity and tight integration with the Apollo platform may limit its flexibility for use in non-Apollo environments.

While general-purpose simulators offer broad applicability across various stages of autonomous driving development, specialized simulators target specific areas, providing high-fidelity environments for functions like vehicle dynamics, traffic flow, or sensor modeling. Together, they create a comprehensive ecosystem of tools supporting the entire autonomous vehicle development process. However, these simulators typically require complex configurations and significant hardware resources, and they offer limited test scenarios, making them less suitable for the data-driven testing and training needed for autonomous decision-making and planning algorithms.

2.2. Scenario-based automated test systems

In addition to general-purpose and specialized simulators, scenario-based automated test systems have become increasingly important for evaluating how autonomous vehicles handle specific challenges, particularly in planning and decision-making. These systems are designed to automate testing by providing a library of predefined driving scenarios that mimic real-world conditions. Their primary focus is to assess a vehicle's ability to plan and execute smooth, safe maneuvers in response to complex environmental inputs. Platforms like CommonRoad (Althoff et al., 2017), NuPlan (Caesar et al., 2022), ScenarioNet (Li et al., 2023e), and Waymax (Gulino et al., 2023) are at the forefront of this type of testing. These systems enable developers to test planning algorithms under various traffic conditions, road layouts,

and dynamic obstacles. Unlike general simulators, these platforms focus specifically on testing the decision-making and planning of autonomous driving.

CommonRoad (Althoff et al., 2017), developed by the Technical University of Munich, provides a composable benchmark for motion planning, offering a wide range of predefined traffic scenarios. The platform is primarily used to evaluate the performance of planning algorithms in complex environments, such as urban intersections and highway merging scenarios. Its library of real-world scenarios, combined with the ability to customize scenarios, makes it a valuable tool for testing the adaptability and robustness of planning systems. NuPlan (Caesar et al., 2022) is another key platform that enables closed-loop, machine-learning-based testing of planning algorithms. The system uses real-world data from various cities to create diverse scenarios, allowing for an iterative testing process where the vehicle's actions affect future conditions. This closed-loop approach provides a more realistic testing environment compared to static scenario-based tests, offering deeper insights into how planning systems will perform in unpredictable, real-world driving conditions. ScenarioNet (Li et al., 2023e) and MetaDrive (Li et al., 2023d) focus on large-scale traffic simulation and modeling, enabling the testing of autonomous vehicles in highly dynamic and complex environments. It emphasizes scalability and allows for the simulation of large numbers of vehicles and pedestrians, making it ideal for testing both individual vehicles and fleets of autonomous vehicles in various traffic situations. This platform is precious for evaluating planning algorithms that simultaneously handle interactions with multiple dynamic agents.

Scenario-based testing systems offer a structured approach to validating the decision-making capabilities of autonomous vehicles by providing a range of predefined and customizable scenarios. These platforms enable developers to rigorously test how vehicles navigate complex environments and respond to unexpected situations. The automation of scenario generation and testing also streamlines the evaluation process, reducing time and effort in algorithm development and making these systems crucial to autonomous driving research. Many platforms, such as ScenarioNet, are built on real-world data and incorporate motion prediction datasets like NuScenes Dataset (Caesar et al., 2020), Waymo Open Motion Dataset (Ettinger et al., 2021), and Lyft Level 5 Prediction Dataset (Houston et al., 2021), providing extensive data support for reinforcement learning (RL), imitation learning (IL), and End-to-End driving task, thus fostering new research opportunities. Meanwhile, these systems have introduced scenario generation as a new research challenge. Waymo's Waymax simulator (Gulino et al., 2023), for example, uses the Waymo Open Motion Dataset to simulate a diverse range of multi-agent scenarios, and it also launched the *Open Sim Agents Challenge 2023* (Montali et al., 2024) to explore data-driven approaches for generating more realistic test scenarios. Scenario generation methods (Feng et al., 2023a; Mo et al., 2024; Wang et al., 2023) typically drive target agents to generate their state in the next step, similar to trajectory prediction tasks, facilitating closed-loop simulation. This is crucial for planning tasks that require extensive scenario testing to validate their robustness. Recent work (Feng et al., 2023b) further explores the use of dense reinforcement learning to enhance the safety validation of autonomous vehicles in complex scenarios, offering new insights into scenario generation methodologies.

2.3. Autonomous driving simulation in open-pit mines

Most existing simulation systems focus on urban driving scenarios, with only a few previous works exploring autonomous driving in open-pit mines environments. For example, OpenMines (Meng et al., 2024) is an open-source framework released to model mining systems, develop scheduling algorithms for mining fleets, and assess their performance. AutoMine (Li et al., 2022b) is the first publicly available perception dataset for open-pit mines environments, but it only supports open-loop testing for perception and prediction tasks and cannot be used for

planning task validation that requires closed-loop simulations. Bai Li et al. (2023a) abstracted loading and unloading transport tasks in open-pit mines as parking tasks in unstructured road scenarios. In 2023, they organized a trajectory planning competition for automated parking (TPCAP), providing reproducible benchmark algorithms and unstructured road scenarios data. These open-source efforts focusing on scheduling, perception, and parking planning tasks in open-pit mines are commendable, but there is still a lack of comprehensive scenario-based testing tools for obstacle avoidance planning algorithms in open-pit mines road scenarios.

Several notable studies have explored planning tasks in open-pit mines, offering valuable insights and inspiration for advancing research in this domain. Usami et al. (2020) proposed a two-lane path planning algorithm for autonomous mining trucks operating in open-pit mines with two-and-a-half-dimensional (2.5D) road environments. Their work considered terrain variations with large slopes and high curvature, primarily focusing on offline High-Definition map path generation. Focusing on loading and dumping scenarios in open-pit mines, Li et al. (2023b) developed a trajectory planning method for parking tasks involving a single heavy-duty mining truck. This method, based on a mixed-integer nonlinear program (MINLP) with conditional constraints, addressed vehicle kinematics and complex collision-avoidance constraints. Meanwhile, Li et al. (2024a) tackled the challenge of planning for multiple mining trucks simultaneously in loading and dumping scenarios, proposing a global collaborative optimization approach that combines heuristic search with group priority optimization methods. For obstacle avoidance planning tasks involving static obstacles on curvy roads, Li et al. (2022a) introduced an iterative optimization framework within a Cartesian coordinate system. This framework effectively resolved complex collision-avoidance and nonconvex kinematic constraints, demonstrating strong applicability to high-curvature road environments typical of open-pit mines. However, there has been limited research on obstacle avoidance planning in mixed-traffic scenarios within open-pit mines, particularly concerning the simulation of vehicle interactions on roads with irregular boundaries. Additionally, learning-based planning methods—often trained on extensive urban road scenarios—have demonstrated high generalizability and strong potential for autonomous mining truck applications. However, implementing these methods for open-pit mining requires large volumes of real-world scenario data specific to mining environments.

To address this gap, MineSim introduces an open-source, closed-loop simulation system specifically designed for autonomous driving planning tasks in open-pit mining. MineSim provides a diverse range of real-world scenarios that capture the unique characteristics of open-pit mines, including: (a) irregular, unsignalized intersections, often with slopes; (b) roads without clear lane markings, irregular boundaries, and varying widths; and (c) large, heavy mining trucks with substantial inertia. These features make MineSim an essential tool for advancing research on autonomous driving in non-urban and unstructured road environments. Notably, the scenario data and testing tools provided by MineSim have been integrated into the Open Naturalistic Simulation and Testing Environment (OnSite, www.onsite.com.cn), a public platform for online autonomous driving testing developed by Tongji University. Furthermore, MineSim's dynamic obstacle scenarios were incorporated as part of the 2024 OnSite Autonomous Driving Algorithm Challenge. Organized by Tongji University and co-hosted by the National Intelligent Connected Vehicle Innovation Center in China, the challenge attracted over 300 teams from more than 70 universities and institutions worldwide (Jiang et al., 2024).

3. System design of MineSim

MineSim system offers a simulation testing environment for motion planning tasks in open-pit mining. It includes both dynamic and static scenario libraries, built from real-world driving data collected from mining sites. Users can evaluate proposed motion planning algorithms

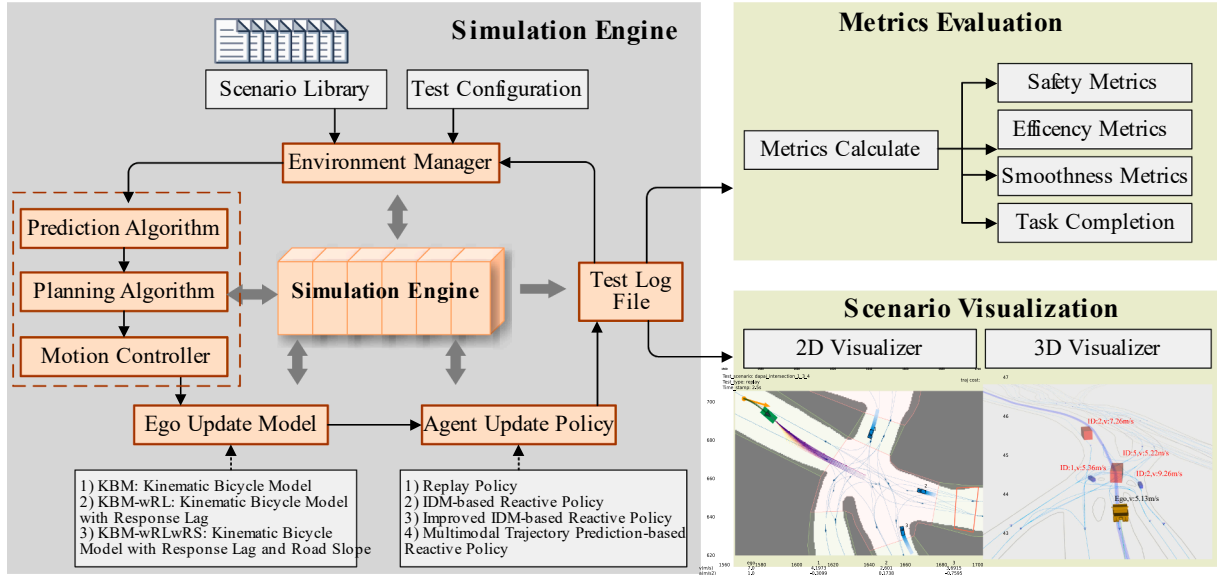


Fig. 1. System architecture of MineSim. MineSim is a scenario-oriented closed-loop simulation testing system, consisting of three components: the Simulation Engine, the Metric Evaluation, and the Scenario Visualization. Herein, IDM refers to the rule-based Intelligent Driver Model (IDM), typically used for traffic flow simulation.

```
{
  "version": "1.1",
  "make_date": "2024-03-01",
  "scenario_info": {
    "scenario_name": "dapai_intersection_1_2_18", "scenario_type": "intersection_mixed", "dt": 0.1,
    "link_map": {
      "map_name": "guangdong_dapai", "version": "1.5"
    },
    "dynamic_obstacle": [
      {
        "id": 2, "StartTimeInScene": 1.5, "EndTimeInScene": 25.4,
        "VehicleShapeInfo": {
          "vehicle_type": "PickupTruck_GreatWall_POER", "length": 5.416, "width": 1.947,
          "height": 1.886, "locationPoint2Head": 2.708, "locationPoint2Rear": 2.708
        },
        "states": {
          "x": [902.495657, 902.675347, ...], "y": [...], "x": [...], "yaw_rad": [...], "v_mps": [...], "a_mps": [...],
          "yawrate_radps": [...],
          ...
        }
      },
      ...
    ],
    "static_obstacle": [
      {
        "id": 101, "is_virtual": "False", "type": "parkedVehicle", "vertices": [...],
        ...
      },
      ...
    ],
    "planning_problem": {
      "ego_info": {
        "vehicle_type": "MineTruck_XG90G", "length": 9, "width": 4, "height": 3.5,
        "min_turn_radius": 12, "wheel_base": 5.15, "locationPoint2Head": 6.5, "locationPoint2Rear": 2.5
      },
      "start_state": {
        "x": 2100.02, "y": 807.968, "yaw_rad": 1.1837, "v_mps": 4.1, "yawrate_radps": 0, "acc_mps": 0
      },
      "goal_state": {
        "goal_box": {
          "x": [...], "y": [...], "goal_pose": [2161.9, 808.3, -2.1407]
        },
        "max_time": 300
      }
    }
  }
}
```

Fig. 2. JSON file structure for each scenario. Note that some elements are omitted for clarity.

for autonomous mining trucks by testing them in numerous scenarios. Fig. 1 illustrates the architecture of MineSim simulation testing system. This system standardizes the representation of mining truck driving scenarios and facilitates large-scale testing of planning algorithms.

MineSim consists of three main components: the *Simulation Engine*, the *Metric Evaluation* component, and the *Scenario Visualization* tool. The *Simulation Engine* serves as the core of MineSim. It loads and parses scenarios from the scenario library via the *Environment Manager*, then implements closed-loop simulation tests for autonomous mining trucks through the sequential execution of the *Prediction Algorithm*, *Planning Algorithm*, *Motion Controller*, *Ego Update Model*, and *Agent Update Policy*. The *Ego Update Model* is responsible for updating the state of the ego vehicle in the simulation. It receives actions output by the tested algorithm, represented as [acceleration, steering angle], and generates the ego vehicle's updated state for the next step. The *Agent Update Policy* uses multiple agent objects and environmental states as inputs to determine the appropriate actions and next state for controllable agents within the scenario. The *Metric Evaluation* component enables users to

create a customized evaluation framework focused on specific performance after a set of scenario tests. It includes four main categories: safety, efficiency, smoothness, and task completion. The *Scenario Visualization* tool provides both 2D top-down and 3D views of the test scenarios, helping users better understand and demonstrate the proposed algorithms' performance.

In the following sections, we will detail each component of MineSim system.

3.1. Unified scenario description in MineSim

The data sources of test scenarios fall into two categories: scenarios converted from real-world logs and virtually generated scenarios. A unified scenario description is essential for consistent storage. As shown in Fig. 2, we define the unified scenario description format as a JSON file. A complete scenario description consists of three main parts: 1) The descriptions of static and dynamic obstacles within the scenario; 2) The setup of the scenario's planning problem; 3) The High-Definition (HD)

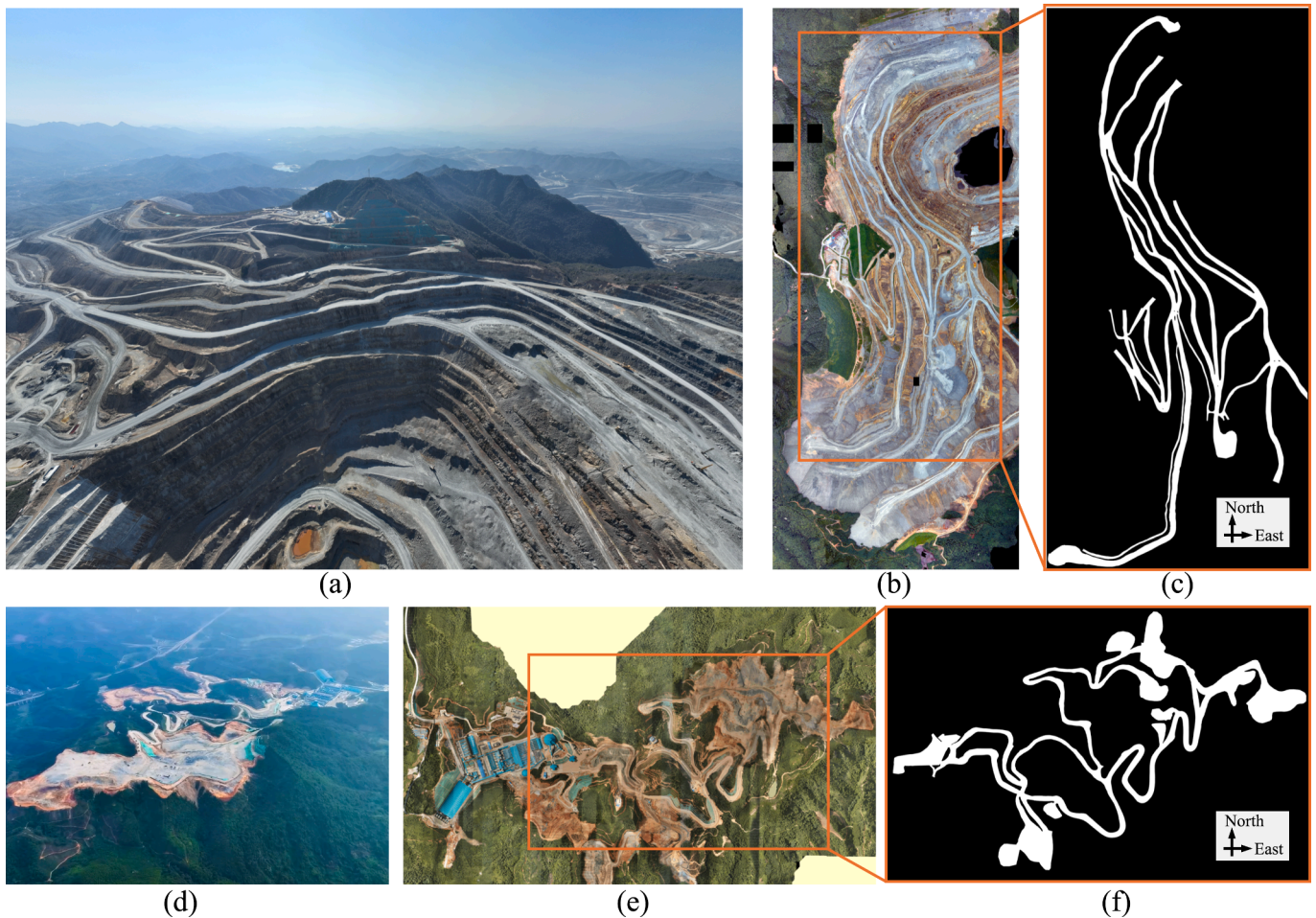


Fig. 3. Real open-pit HD map of unstructured roads in MineSim. (a-c) Mines snapshot, orthophoto image, and raster map at Jiangtong open-pit, Jiangxi Province, China; (d-f) Mines snapshot, orthophoto image, and raster map at Dapai open-pit, Guangdong Province, China.

Map of the corresponding area for the scenario.

3.1.1. Traffic agents in mining scenarios

Unlike the complex and open urban roads filled with various cars, bicycles, and pedestrians, the primary traffic participants in open-pit mines are autonomous mining trucks and various static and dynamic obstacles. Following common practices from open-source autonomous driving datasets (Althoff et al., 2017; Caesar et al., 2022; Ettinger et al., 2021), traffic participants are stored in a time-frame-based format. The “dynamic obstacles” field contain complete trajectory information, and any frame drop issues are addressed during the data processing stage. The “static obstacles” field stores details such as obstacle type (e.g., parked vehicles, rocks, construction areas, unknown), bounding box coordinates, and whether the obstacle is virtually generated. The ego vehicle’s parameter information is stored in the “planning problem” field. In addition to ego parameters, this field also includes the ego vehicle’s start state, goal state, and the maximum allowed runtime for the test task. Based on the scenario descriptions above, MineSim supports automated testing for various planning tasks of autonomous mining vehicles in open-pit mines. Two clearly defined scenario libraries are provided:

- (1) Dynamic obstacle scenario library: This library focuses on testing obstacle avoidance in mixed traffic at mining intersections. Its scenarios feature unstructured intersections with varying slopes and irregular shapes, where autonomous mining trucks interact with other vehicles. The main goal is to assess the trucks’ smooth

obstacle avoidance. Task settings and benchmarks are detailed in Section 4.

- (2) Static obstacle scenario library: This library is designed to test obstacle avoidance with static obstacles on mining roads. It records scenarios where the trucks face static obstacles of various sizes on roads with different curves and widths. The focus is on assessing the ability of autonomous mining trucks to plan smooth, efficient trajectories. Detailed task settings and benchmarks are provided in Section 5.

3.1.2. HD map descriptions for unstructured roads in open-pit mines

The critical differences between unstructured roads in open-pit mines (akin to off-road terrain) and structured roads such as urban highways are the lack of clear lane markings, irregular road boundaries, and uneven road widths. Additionally, these roads often feature steep slopes and frequent gradient changes. As industrial environments, mines have constantly changed operational areas, such as loading and unloading zones, where drivable areas shift frequently. As shown in Fig. 3, these are typical mining roads provided in MineSim. Drawing on an open-source map format (Caesar et al., 2020), we designed an HD map to accurately represent these characteristics of unstructured roads in open-pit mines. The HD map consists of two components: a raster map and a semantic map. The scenario description file in Fig. 2 contains a “link_map” field that stores the map information covering the entire area. The *Simulation Engine* automatically indexes the corresponding HD map file, which is described in a separate JSON file.

A. Raster Map: We use high-resolution mask map data to represent

the drivable areas of mines roads. This format preserves the maximum detail for all drivable areas within the entire area, as illustrated in Fig. 3 (c) and 3(f). In the raster map, each pixel contains a value of either 0 or 1, indicating non-drivable and drivable areas, respectively. The pixel coordinates are precisely mapped to absolute geographical coordinates, with a scale of 10 pixels per meter.

B. Semantic Map: The high-precision semantic map consists of geometric and semantic layers within the entire area, with precise location data for each layer. The *geometric layers* include “node,” “node_block,” and “polygon,” recording various geometric elements like points and surfaces that compose the semantic map. The *semantic layers* represented by geometric polygons include “road,” “intersection,” “loading_area,” “unloading_area,” and “road_block.” These layers divide the entire mines’s drivable areas into smaller regions such as road segments, intersections, loading areas, and unloading areas, each associated with potential traffic conflicts. The *semantic layers* represented by geometric lines include “dubins_pose,” “reference_path,” and “borderline.” Both the *geometric* and *semantic layers* are managed by a unique identifier called a “token,” ensuring the uniqueness of each element within the map system. More detailed map information can be found in Appendix A.

3.2. Ego vehicle state update

The ego vehicle’s state update model (also referred to as the vehicle dynamics model) has been extensively studied in vehicle control (Rajamani, 2012). MineSim is primarily designed to test motion planning tasks of autonomous mining trucks. Therefore, a high-fidelity vehicle state update model is not required. The commonly used Kinematic Bicycle Model (KBM) is employed. Considering the lag response characteristics of heavy mining trucks’ driving, braking, and steering systems, the Kinematic Bicycle Model with Response Lag (KBM-wRL) is proposed. Additionally, to account for the frequently changing slopes in mining areas, a vehicle–road coupled dynamics update model called the Kinematic Bicycle Model with Response Lag and Road Slope (KBM-wRLwRS) is introduced. This model aims to more accurately capture the state updates of heavy transport trucks on rugged mining roads. All ego update models in this paper are represented as discrete state-space functions:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \quad (1)$$

where $\mathbf{x}_k$ represents the ego vehicle’s state at the k -th time step, and $\mathbf{u}_k$ represents the ego vehicle’s control input at the k -th time step. In MineSim, the ego vehicle uses a front-wheel-only steering system, and the state update model does not account for tire dynamics or more complex multi-body vehicle dynamics. Overly complex vehicle models do not contribute to the efficiency of our testing system. The constraints of the various state update models introduced in this section, and the vehicle parameters used in the simulation system, are detailed in Appendix B.

3.2.1. Kinematic bicycle model

When investigating motion planning for autonomous mining trucks, the Point-Mass Model (Li et al., 2023d) alone is insufficient in simulations. Factors such as rollover limits, steering limits, and dynamic constraints must also be considered. Therefore, a kinematic bicycle model (also known as the kinematic single-track model) is adopted with input saturation, which simplifies the front and rear wheel pairs into single wheels. Due to the low-speed operation of mining trucks, tire slip and the vehicle’s slip angle are not considered. The kinematic bicycle model’s state-space function is as follows:

$$\mathbf{x}_{k+1} = v_{lon,k} \cdot \cos(\theta_k) \cdot T \quad (2a)$$

$$y_{k+1} = v_{lon,k} \cdot \sin(\theta_k) \cdot T \quad (2b)$$

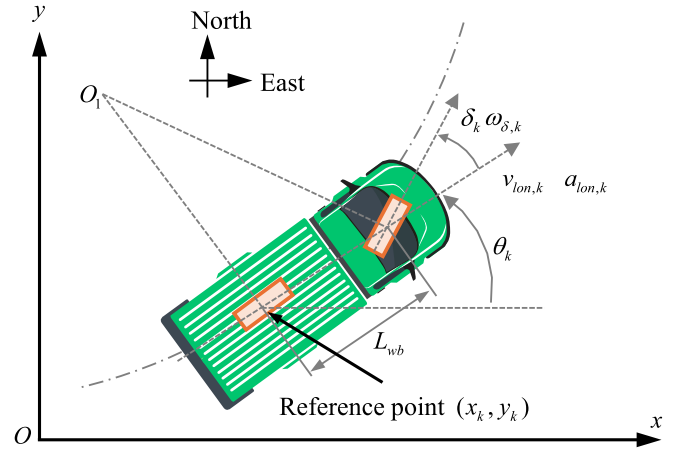


Fig. 4. Schematic of the kinematic bicycle model.

$$\theta_{k+1} = \theta_k + \frac{v_{lon,k} \cdot \tan(\delta_k)}{L_{wb}} \cdot T \quad (2c)$$

$$v_{lon,k+1} = v_{lon,k} + a_{lon,k} \cdot T \quad (2d)$$

$$\delta_{k+1} = \delta_k + \omega_{\delta,k} \cdot T \quad (2e)$$

where T represents the discrete time step set in the simulation. The ego vehicle’s states $\mathbf{x}_k = [x_k, y_k, \theta_k, v_{lon,k}, \delta_k]$ represent the eastward global position, northward global position, yaw angle, longitudinal velocity, and front wheel steering angle, respectively. The control inputs $\mathbf{u}_k = [a_{lon,k}, \omega_{\delta,k}]$ denote the longitudinal acceleration and steering angular rate as output by the planning or control algorithm. L_{wb} represents the vehicle’s wheelbase. The steering angle updated by Eq. (2e) must be clipped as $\delta_{k+1} = \text{clip}(\delta_{k+1}, \delta_{\min}, \delta_{\max})$. The meanings of the various vehicle symbols are illustrated in Fig. 4.

3.2.2. Kinematic bicycle model with response lag

Heavy mining trucks typically used for transporting ore are characterized by large inertia. Therefore, the response lag in the steering, brake, throttle, and transmission systems cannot be ignored (Xu et al., 2021), as the response to control inputs is not instantaneous. Accordingly, a first-order linear system model is used to approximate this response lag. The vehicle’s state-space function is as follows:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_{2,k}) \quad (3a)$$

$$\mathbf{u}_{2,k} = f_{lag}(\mathbf{u}_{1,k}) \quad (3b)$$

where the ego vehicle’s states are still represented by $\mathbf{x}_k = [x_k, y_k, \theta_k, v_{lon,k}, \delta_k]$. $\mathbf{u}_{1,k} = [a_{lon,ideal,k}, \omega_{\delta,ideal,k}]$ represents the longitudinal acceleration and steering angular rate as output by the planning or control algorithm. $\mathbf{u}_{2,k} = [a_{lon,update,k}, \omega_{\delta,update,k}]$ denotes the longitudinal acceleration and steering angular rate after being updated through the lag process. The vehicle’s state-space function is described as follows:

$$\mathbf{x}_{k+1} = v_{lon,k} \cdot \cos(\theta_k) \cdot T \quad (4a)$$

$$y_{k+1} = v_{lon,k} \cdot \sin(\theta_k) \cdot T \quad (4b)$$

$$\theta_{k+1} = \theta_k + \frac{v_{lon,k} \cdot \tan(\delta_k)}{L_{wb}} \cdot T \quad (4c)$$

$$v_{lon,k+1} = v_{lon,k} + a_{lon,update,k} \cdot T \quad (4d)$$

$$\delta_{k+1} = \delta_k + \omega_{\delta,update,k} \cdot T \quad (4e)$$

The control inputs incorporate a lag process based on a first-order linear system as:

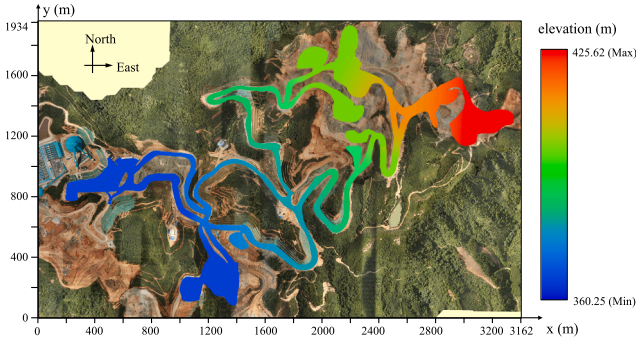


Fig. 5. Elevation reconstruction corresponding to the raster map.

$$a_{lon,update,k} = \frac{T}{T + \tau_a} (a_{lon,idea,k} - a_{lon,k}) + a_{lon,k} \quad (5a)$$

$$\delta_{ideal,k} = \delta_k + \omega_{\delta,ideal,k} \cdot T \quad (5b)$$

$$\delta_{update,k} = \frac{T}{T + \tau_{\delta}} (\delta_{ideal,k} - \delta_k) + \delta_k \quad (5c)$$

$$\omega_{\delta,update,k} = \frac{\delta_{update,k} - \delta_k}{T} \quad (5d)$$

where τ_a represents the time constant for the vehicle's longitudinal acceleration. τ_{δ} represents the time constant for the vehicle's steering angle. First, the ideal steering angular velocity is converted to the ideal steering angle. Then, the steering system is modeled as a first-order linear system using Eq. (5c) to simulate the response delay in the steering system.

3.2.3. Kinematic bicycle model with road slope

In open-pit mines, frequent slope changes on the roads require the control and planning algorithms to consider the road's gradient information. Therefore, MineSim system provides slope modeling for all drivable areas. Using a 2D grid map, the elevation information for different points can be easily indexed to simulate the effects of uphill and downhill surfaces on the vehicle's state updates. This model, referred to as the "Vehicle-Road Coupling Dynamics Model," uses the elevation data to calculate the slope between points (Fig. 5 shows the visualized slope modeling corresponding to the Raster Map). The slope is then calculated to adjust the longitudinal acceleration in the models as in Sections 3.2.1 or 3.2.2. For instance, when adjusting the state update model from Section 3.2.2, the rest of the model remains unchanged, and the adjustment to the longitudinal acceleration is as follows:

Calculate the nominal position update:

$$\begin{aligned} x_{nominal,k+1} &= v_{lon,k} \cdot \cos(\theta_k) \\ y_{nominal,k+1} &= v_{lon,k} \cdot \sin(\theta_k) \end{aligned} \quad (6)$$

Use the nominal position change to query the elevation information from the map and compute the slope term:

$$slope_k = \frac{h_{k+1} - h_k}{\sqrt{(x_{nominal,k+1} - x_k)^2 + (y_{nominal,k+1} - y_k)^2}} \quad (7)$$

After adding the lag process to the control inputs, incorporate the slope component based on a first-order linear system as:

$$a_{lon,update,k} = \left(\frac{T}{T + \tau_a} (a_{lon,idea,k} - a_{lon,k}) + a_{lon,k} \right) - g \cdot \sin(slope_k) \quad (8)$$

3.3. States update for other agents

Currently, scenario-based testing tools for autonomous driving (Gulino et al., 2023; Jia et al., 2024; Caesar et al., 2022) typically offer

two types of policies for updating other agent states: Replay Policy (where agent data is sourced from real-world data) and Reactive Policy. The Reactive Policy addresses how multiple agents in a scenario can achieve realistic and reasonable state updates, making it an essential research focus in the field of scenario generation, with some challenges emerging at the research forefront (Bergamini et al., 2021; Feng et al., 2023a). It is a scenario-level state update model that takes the states of multiple agents and the environment as inputs to determine the appropriate actions and the next step for controllable agents. In open-pit mines, other traffic agents primarily refer to obstacle vehicles, and MineSim provides a Replay Policy and three types of Reactive Policies.

- (1) **Replay Policy:** This policy involves replay testing based on logs recorded from real-world scenarios, which are replayed step-by-step according to the simulation cycle.
- (2) **IDM-based Reactive Policy:** This rule-based longitudinal Intelligent Driving Model (IDM) is typically used for traffic flow simulation. It enables behaviors such as cruising, following, and emergency stopping along a predefined path. Additionally, various driving styles, i.e. "conservative," "moderate," and "aggressive" can be configured by adjusting IDM parameters. The IDM in MineSim is designed based on the model described in the reference (Treiber et al., 2000).

$$s^*(v_{agent}, v_{lead}) = s_0 + v_{agent} \cdot T + \frac{v_{agent} \cdot (v_{agent} - v_{lead})}{2 \cdot \sqrt{a \cdot b}} \quad (9)$$

$$\dot{x}_{lon} = v_{agent} \quad (10)$$

$$\dot{v}_{agent} = v_{agent}^{free} + v_{agent}^{interact} = a_{agent} \left(1 - \left(\frac{v_{agent}}{v_0} \right)^{\delta} \right) - a_{agent} \left(\frac{s^*}{s_a} \right)^2 \quad (11)$$

where v_0 is the desired velocity that the vehicle would drive at in free-flowing traffic, s_0 is the minimum safety distance if another vehicle is present ahead, and T is the desired time headway in such situations. a represents the maximum vehicle acceleration, and b is the comfortable braking deceleration. The acceleration exponent, δ , is typically set to 4. The agent's acceleration can be divided into a free-road term and an interaction term. The free-road term governs the vehicle's acceleration on open roads, while the interaction term adjusts the vehicle's behavior based on the distance and speed difference relative to the vehicle ahead.

- (3) **Improved IDM-based Reactive Policy:** In MineSim, we have improved the input strategy of the IDM model. Since the roads in open-pit mines lack clear lane markings, vehicles do not necessarily follow the reference path, which exists only on our map. Therefore, when determining the corresponding lead vehicle for each agent, the logic must consider the predicted information of other agents. The Constant Velocity Constant Yaw Rate (CVCYR) prediction method (Kim et al., 2014) is used to predict the future states of other agents, projecting these states onto the agent's matched path. The agent's state is then updated according to the IDM model.
- (4) **Multimodal Trajectory Prediction-based Reactive Policy:** It is worth noting that another area of focus in scenario generation involves Neural Network-based state update models for single-agent or multi-agent systems. These models are typically scenario-centric and use trajectory prediction networks to simultaneously update the states of individual agents or multiple agents. They enable the simulation of more diverse and realistic behaviors, facilitating the creation of more generalized and authentic closed-loop traffic scenarios. This is a key advantage over traditional IDM-based methods, which primarily focus on longitudinal behavior and are unable to capture the full range of agent interactions. However, these trajectory prediction models have notable drawbacks, particularly in real-time simulations involving a large number of agents. They require significant

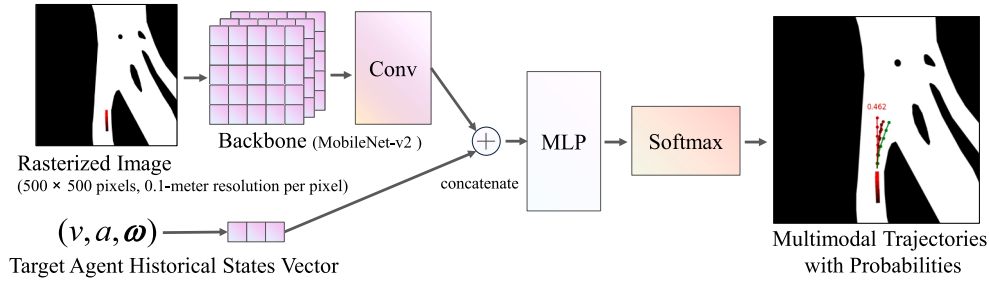


Fig. 6. Principle of agent states update via the Multimodal Trajectory Prediction-based Reactive Policy.

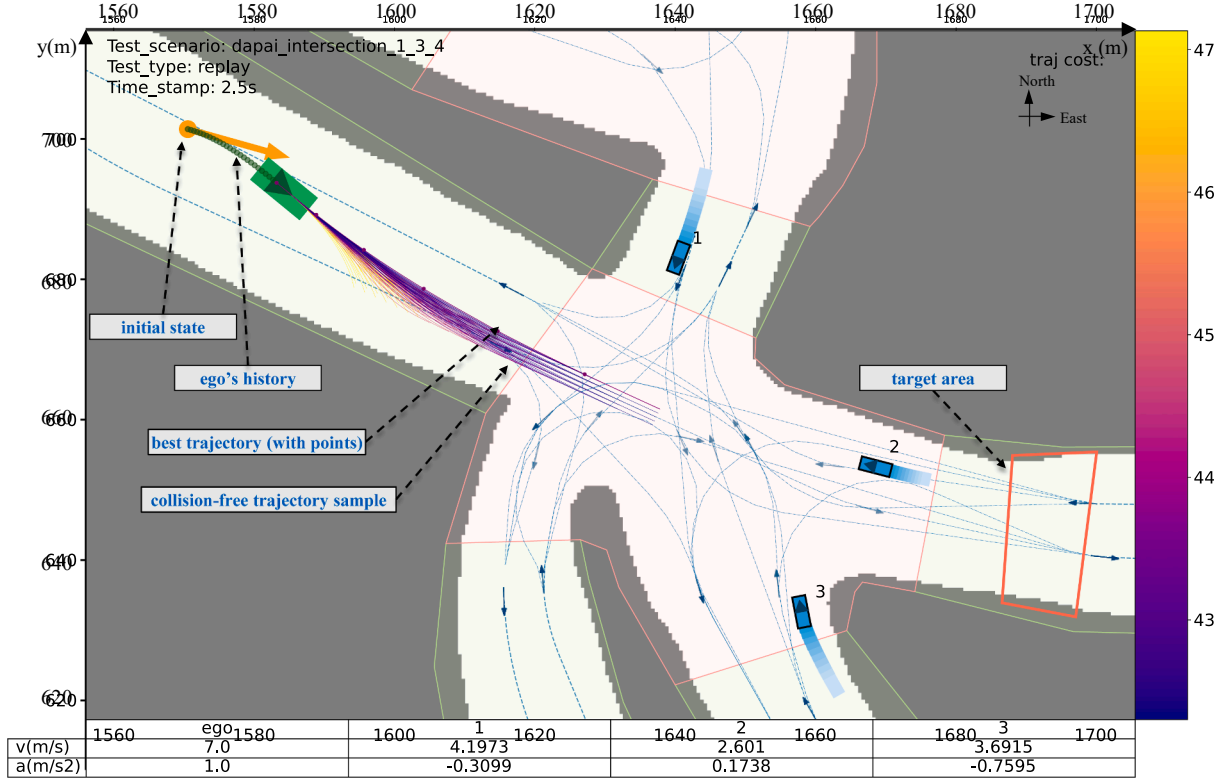


Fig. 7. Example of a dynamic obstacle avoidance task.

computational resources and may occasionally produce rare or difficult-to-interpret simulation results, an issue that warrants further investigation and improvement. Despite these challenges, such models are crucial for the training and testing of Reinforcement Learning or End-to-End autonomous driving algorithms. In MineSim, we provide a lightweight Multimodal Trajectory Prediction model (Li et al., 2024b) to update single-agent states, allowing for more diverse and realistic lateral and longitudinal state updates in mining scenarios. As illustrated in Fig. 6, the architecture of the prediction model uses the target agent's historical 3-second trajectory and renders its shape contour onto a mask raster map. A CNN-based backbone network then encodes the rendered images to extract features, and the model generates k possible trajectories, each with associated probability scores.

3.4. Scenario visualization tool

It is crucial for researchers to quickly iterate on a new planning algorithm and closely investigate the algorithm's performance. Therefore, MineSim provides two Scenario Visualization Tools: the 2D Visualizer

and the 3D Visualizer.

- (1) **2D Visualizer:** Developed using Python and Matplotlib, this tool renders top-down views of each frame of a single scenario based on the simulation log file. Researchers can easily integrate scenario-specific and algorithm-related information to the API as needed, such as the planned trajectory shown in Fig. 11.
- (2) **3D Visualizer:** Built using ROS, this tool renders 3D views of each frame in a scenario based on the simulation log file. It allows for an intuitive observation of vehicle interactions and longitudinal speed planning information for the ego vehicle, as illustrated in Fig. 11.

4. Scenario library and benchmark for dynamic obstacle avoidance at mining intersections

Open-pit mines are characterized by numerous irregular, unsignalized intersections, often on sloped roads. Therefore, autonomous mining trucks frequently operate together with human-driven trucks or cars, creating complex and challenging mixed-traffic scenarios. Effective obstacle avoidance planning under such mixed-traffic conditions is

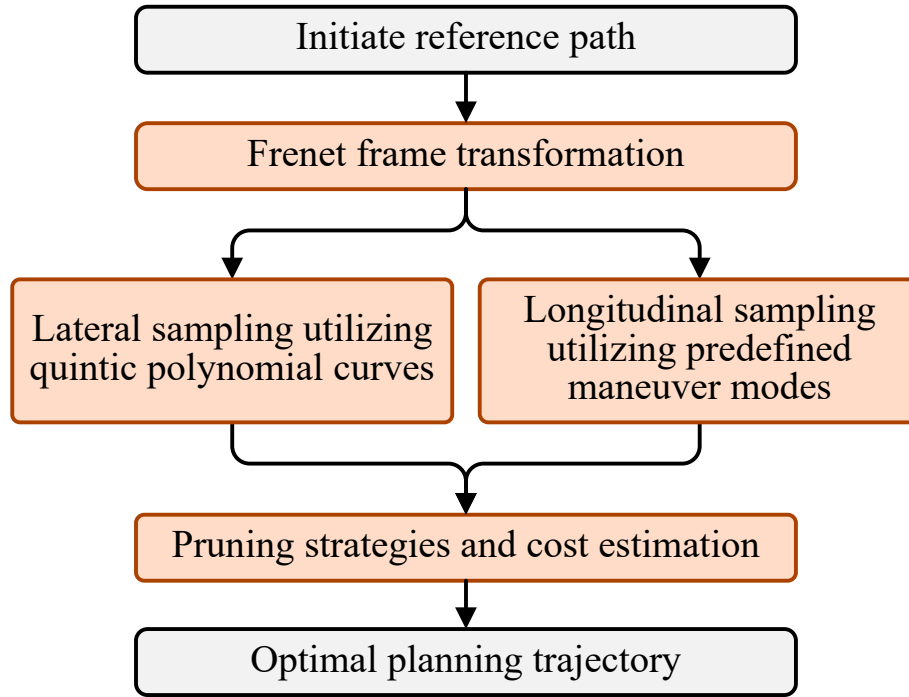


Fig. 8. Architecture of the SPPMM algorithm.

crucial for smooth, efficient, and safe operations. MineSim provides a scenario library (named as “dataset-dynamicsscenario-v1.0”) designed to test obstacle avoidance in mixed traffic at mining intersections. It includes 210 scenarios derived from real-world data.

4.1. Task description

The dynamic obstacle avoidance planning problem is first defined for each scenario. A complete test task includes the vehicle parameters, the ego vehicle’s state update model, the initial states, the target states, and the other agents’ state update policy. As shown in Fig. 7, the ego vehicle’s reference point is the rear axle center, with the initial state (denoted as $\mathbf{x}_0 = [x_0, y_0, \theta_0, v_0, \gamma_0, a_0, \delta_0]$, representing x and y positions, yaw, speed, yaw rate, acceleration, and front wheel steering angle) indicated by a yellow arrow. The target area, marked by an orange polygon, represents the region the vehicle must navigate to complete the task.

4.2. Metrics description

To comprehensively evaluate the planning algorithm’s performance across various scenarios, our scoring system assesses four key categories of metrics: safety, efficiency, smoothness, and task completion. These metrics are aggregated to produce an overall score for the proposed algorithm, providing a holistic assessment of its capabilities.

- (1) **Safety Metrics:** This metric evaluates the vehicle’s ability to avoid collisions with obstacles and road boundaries. Given the irregular characteristics of unstructured roads, the vehicle must maintain a sufficient safety margin from the boundaries. Specifically, the lateral distance between the ego vehicle and the road boundary must exceed a predefined threshold to ensure safety throughout the scenario.
- (2) **Efficiency Metrics:** Efficiency is measured by how quickly the autonomous vehicle reaches the target area, provided it completes the scenario safely. Faster completion times indicate better performance.

- (3) **Smoothness Metrics:** Unlike passenger vehicles, where algorithms focus on optimizing ride comfort based on passengers’ subjective experience, the primary goal for mining vehicles is driving smoothness. This metric is assessed by calculating the minimum and maximum longitudinal and lateral accelerations, as well as centripetal acceleration during turns to evaluate stability. Smoothness is crucial for ensuring stability and safety in challenging mining environments.
- (4) **Task Completion:** If the target is not reached, the completion score is based on the vehicle’s progress along the route, measured by the ratio of the distance traveled to the total route distance.

When testing the performance of the provided algorithm across a wide range of scenarios, key metrics like Task Completion Rate and Collision Rate can also serve as important indicators, depending on the metrics prioritized.

4.3. Benchmark algorithm and test results

4.3.1. Benchmark algorithm

For the obstacle avoidance problem in scenarios with dynamic obstacles, we provide two benchmark algorithms: the IDM introduced in Section 3.3, and a Sampling Planner based on Predefined Maneuver Modes (SPPMM). The overall architecture of the SPPMM algorithm, as shown in Fig. 8, represents an improvement upon the Frenet-based Optimal Planner (FOP) (Werling et al., 2010). The FOP uses the Frenet frame for trajectory planning, decoupling the process into longitudinal and lateral components. Each trajectory is created by combining two high-order polynomials, one for each direction. After generating multiple trajectory samples, those that violate dynamic or safety constraints are pruned, and the remaining trajectories are evaluated to select the optimal one. Our approach follows the FOP method, with modifications specifically to the longitudinal sampling strategy.

Several other methods have made improvements to the FOP. Baidu’s Lattice Planner (Fan et al., 2018) increased planning speed by reducing the number of collision checks. Sun et al. (2022, 2023) introduced the Fast Iterative Search and Sampling (FISS) strategy, which iteratively refines the planned trajectory by sampling multiple times at the terminal

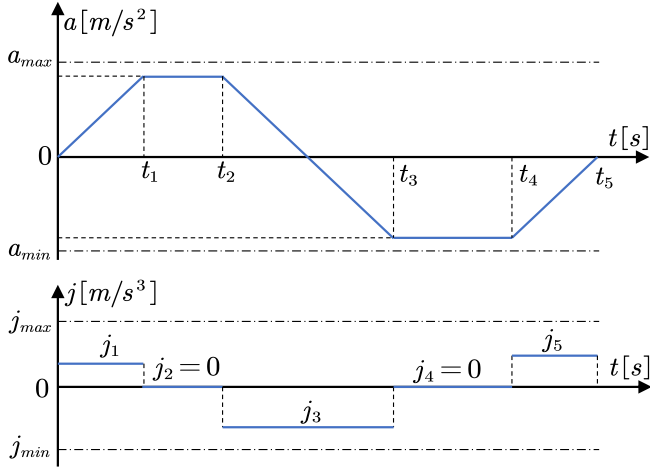


Fig. 9. Double-trapezoid curve in the a-t space.

state. However, those parametric curve-based sampling methods implicitly assumes that the ego vehicle can only motion in a single mode—constant acceleration, constant deceleration, or constant speed—during the planning horizon. The insufficient exploration of the acceleration-time solution space, especially for the critical next-time steps, significantly impacts the subsequent behavior of an autonomous vehicle, as shown in Fig. 10. Therefore, the motivation behind SPPMM is to develop a probabilistically complete sampling approach based on predefined maneuver modes.

The predefined maneuver modes-based sampling method operates under a key assumption: “The ego vehicle’s reasonable longitudinal trajectory within a limited planning horizon can generally be represented by a finite set of continuous maneuver modes.” This assumption is particularly relevant for obstacle avoidance, where combining different longitudinal maneuver modes yields efficient and probabilistically complete trajectory sets. Additionally, it helps mitigate the “Dimensionality Curse” problem often encountered in high-dimensional sampling processes. To implement this assumption, we propose a generalized formulation called the “Double-Trapezoid Curve in the a-t Space”, implemented as follows:

- (1) **Predefined Curve Combinations:** We assume that during future planning horizon (typically 5 to 10 s), the ego vehicle can follow no more than two maneuver modes. The simplest vehicle maneuver can be categorized into three types: acceleration ($a > 0$), deceleration ($a < 0$), and constant speed ($a = 0$), resulting in nine possible combinations: 1) continuous acceleration; 2) acceleration followed by deceleration; 3) acceleration followed by constant speed; 4) continuous deceleration; 5) deceleration followed by acceleration; 6) deceleration followed by constant speed; 7) continuous constant speed; 8) constant speed followed by acceleration; 9) constant speed followed by deceleration. To generalize these maneuver mode combinations, we propose a predefined Double-Trapezoid Curve in the a-t Space, as illustrated in Fig. 9.

- (2) **Sampling, Integration, and Constraint-Check Process:** The sampling strategy of predefined maneuver modes is formulated as shown in Eqs. (12), (13) and (14). First, we divide the planning horizon into five continuous intervals for constant jerk sampling, following the rules outlined in Eq. (12) to generate multiple seed $= (t_1, j_1, t_2, j_2, t_3, j_3, t_4, j_4, t_5, j_5)$. Then, based on the real-time state values s_0, v_0 , and a_0 , we use numerical integration, as shown in Eq. (13), to determine s_1, v_1 , and a_1 at the end of the first time interval. The same procedure is applied to the remaining time intervals. As demonstrated in Eq. (14), the sampled seeds are also subjected to kinematic constraint checks after integration for each time interval; any seeds that exceeding the constraints are discarded.

$$\begin{cases} t_1, t_2, t_3, t_4, t_5 \in \mathbb{T} = \{t | t \in [0, \Delta t, \dots, T_{\text{plan}}]\} \\ t_1 + t_2 + t_3 + t_4 + t_5 = T_{\text{plan}} \\ j_1, j_3, j_5 \in \mathbb{J} = \{j | j \in [j_{\min}, j_{\min} + \Delta j, \dots, j_{\max}]\} \\ j_2 = j_4 = 0 \end{cases} \quad (12)$$

$$\begin{cases} a_1 = j(t_1 - t_0) + a_0 \\ v_1 = \frac{1}{2}j(t_1 - t_0)^2 + a_0(t_1 - t_0) + v_0 \\ s_1 = \frac{1}{6}j(t_1 - t_0)^3 + \frac{1}{2}a_0(t_1 - t_0)^2 + v_0(t_1 - t_0) + s_0 \end{cases} \quad (13)$$

$$\begin{cases} a_{\min} \leq a_1, a_2, a_3, a_4, a_5 \leq a_{\max} \\ 0 \leq v_1, v_2, v_3, v_4, v_5 \leq v_{\max} \\ s_2 - s_1, s_3 - s_2, s_4 - s_3, s_5 - s_4 \geq 0 \end{cases} \quad (14)$$

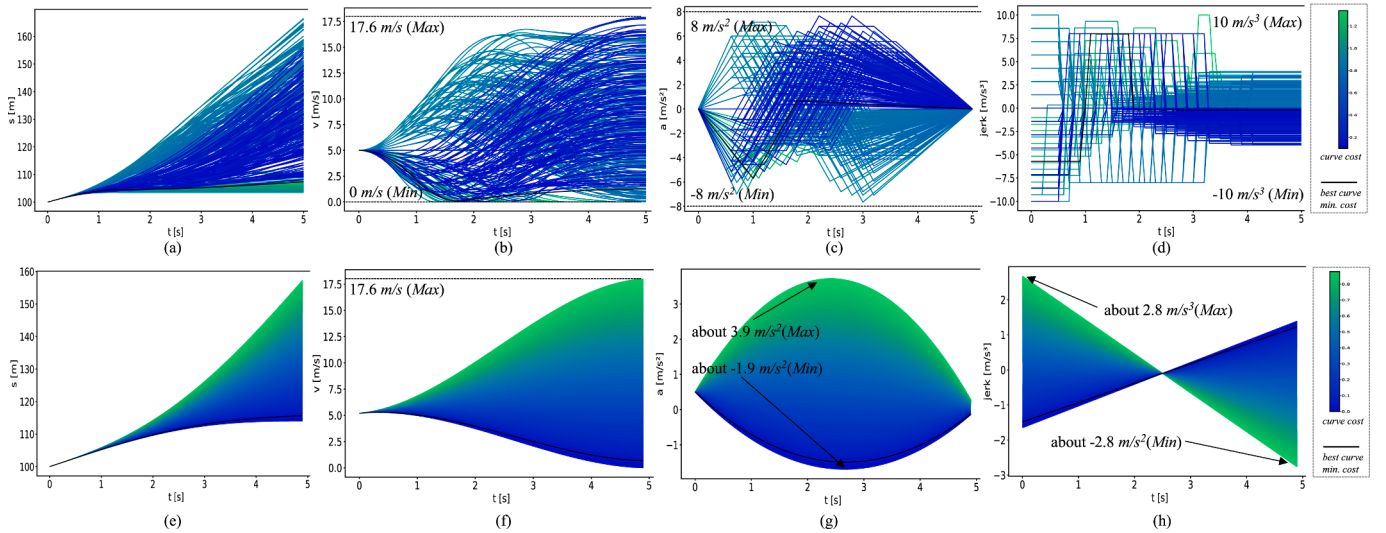


Fig. 10. Sampling results of different longitudinal sampling strategies. The number of samples was set at 839. (a-d) show sampling results based on predefined maneuver modes, while (e-h) show results based on quintic polynomials. The initial sampling state is set with a velocity of 5.2 m/s and an acceleration of 0.5 m/s², subject to constraints of velocity between 0.0 and 18 m/s, acceleration between -8 and 8 m/s², and jerk between -10.0 and 10.0 m/s³.

Table 1

Metric design for dynamic obstacle avoidance scenario testing.

Metric Categories	Metric Name	Metric Score Calculation Description	Points
Safety (40 points)	Collisions with Obstacles	Penalty: Safety score is 0 if a collision occurs.	0 or 40
	Collisions with Road Boundaries	Penalty: Safety score is 0 if a collision occurs.	0 or 40
	Lateral Safety (Roadside)	Penalty: Deduct points based on the duration below the lateral safety distance threshold over total driving time.	0–40
Efficiency (10 points)	Driving Duration	If the safety score is 0, this score is also 0; otherwise, score is calculated as $\min(1.0, \text{total driving time} / \text{expected time}) * 10$.	0–10
Smoothness (30 points)	Lateral	Penalty: $\min(1.0, \text{duration exceeding lateral acceleration threshold} / \text{total driving time}) * 10$.	0–10
	Longitudinal	Penalty: $\min(1.0, \text{duration exceeding longitudinal acceleration threshold} / \text{total driving time}) * 10$.	0–10
	Turning	Penalty: $\min(1.0, \text{duration exceeding centripetal acceleration threshold} / \text{total driving time}) * 10$.	0–10
Task Completion (20 points)	Progress along Route Path	If the ego vehicle reaches the goal, the score is 20; otherwise, it is calculated as $\min(1.0, \text{final driving distance along the route path} / \text{overall route path distance}) * 20$.	0–20

Table 2

Quantitative experimental score results for dynamic obstacle avoidance scenario testing.

Algorithms	Safety	Efficiency	Smoothness	Task completion	Total Score
IDM	32.5	9.3	23.5	19.3	84.6
SPPMM	28.5	8.2	25.8	18.6	81.1

where Δt is the planning time step, T_{plan} is the planning horizon, $\mathbb{J}$ is the jerk sampling space, and Δj is interval for uniform jerk sampling.

4.3.2. Test results of dynamic obstacle avoidance benchmark

- (1) Experiment Setup: We evaluated two benchmark algorithms for the dynamic obstacle avoidance task, which were automatically tested using 210 scenarios. The simulation engine mode was configured as *Replay Test Mode*, which corresponds to ego closed-loop with non-reactive agents (one can refer to Appendix C for all configurable methods). The prediction algorithms were set to perfect prediction, the ego closed-loop simulation was configured with the LQR-based motion controller and KBM-wRL ego update model, and the agent simulation used the replay policy.
- (2) Quantitative Analysis: Utilizing the metrics designed in Table 1, the proposed benchmarks were comprehensively evaluated with the quantitative experimental results presented in Table 2. The IDM and the SPPMM methods exhibit strengths and weaknesses across different metrics. The IDM method adopts a longitudinal speed adjustment strategy along the reference path, performing better in safety and task completion. In contrast, the SPPMM method, which employs lateral and longitudinal sampling for obstacle avoidance, shows better performance in efficiency and smoothness due to the polynomial curves used to parameterize the trajectory, which better aligns with vehicle kinematics. However, the SPPMM method performs worse regarding safety,

with more collisions observed. Future improvements could focus on enhancing safety distance concerning other obstacles.

- (3) Qualitative Analysis: Both 2D and 3D visualizers were employed to demonstrate the obstacle avoidance processes in two test scenarios (see Fig. 11 and Fig. 12). As shown on the right column of Fig. 11(a), collision-free trajectory samples are depicted by various colored lines, each representing a trajectory that has successfully passed collision checks and dynamics constraints. The optimal trajectory, marked by distinct points, indicates the optimal trajectory determined through cost evaluation. By comparing Fig. 11(g) and (h), it is visually apparent that the SPPMM method reaches the target area more quickly than the IDM method.

5. Scenario library and benchmark for static obstacle avoidance in mines

Open-pit mines are typical production environments where mining trucks primarily transport ore or debris. Roads in these areas are often obstructed by static obstacles such as parked vehicles or large rocks along the roadside. Effective obstacle avoidance for autonomous mining trucks is essential to ensure safe and efficient operations in these environments. MineSim provides a scenario library (named as “dataset-staticscenario-v1.0”) focused on static obstacles along mining roads, aiming to evaluate the performance of obstacle avoidance algorithms under varying road curvatures and obstacle placements. It includes 22 static obstacle scenarios based on common challenges in open-pit mines. Both qualitative and quantitative comparisons of two benchmark algorithms are provided.

5.1. Task description

The static obstacle avoidance planning problem is first defined for each scenario. A complete test task consists of the ego vehicle's parameters, state update model, initial state, and target state. As shown in Fig. 13, this task differs from the dynamic scenario task design (Fig. 7) by incorporating a desired final pose, $\text{pose}_{\text{Goal}} = [x_G, y_G, \theta_G]$, indicated by the red arrow. After completing the obstacle avoidance task, the vehicle is expected to return close to the reference path with proper alignment.

5.2. Metrics description

The metrics design for the static scenario planning task is shown in Table 3 and differs slightly from the Task completion metrics used for the dynamic scenario planning task. Task completion is evaluated for both scenarios in which the vehicle reaches the target area and those in which it does not. If the target is not reached, the completion score is determined by the vehicle's progress along the route, specifically by comparing the final driving distance to the total route distance. If the vehicle reaches the target, the quality of task completion is also assessed by evaluating the final yaw deviation, which measures the vehicle's alignment with the road at the goal based on the yaw difference from the target orientation.

5.3. Benchmark algorithm and test results

5.3.1. Benchmark algorithm

For the obstacle avoidance problem in scenarios with only static obstacles, two benchmark algorithms were provided: one is the SPPMM as introduced in the 4.1 section, and the other is a two-stage planner that combining Hybrid A* with Constrained iterative Linear Quadratic Regulator (HA*-CiLQR). The HA*-CiLQR algorithm integrates a coarse, collision-free path generated by a graph search-based front-end with a smooth trajectory refined through optimal control in the back-end, producing an optimized obstacle-avoidance trajectory, as shown in

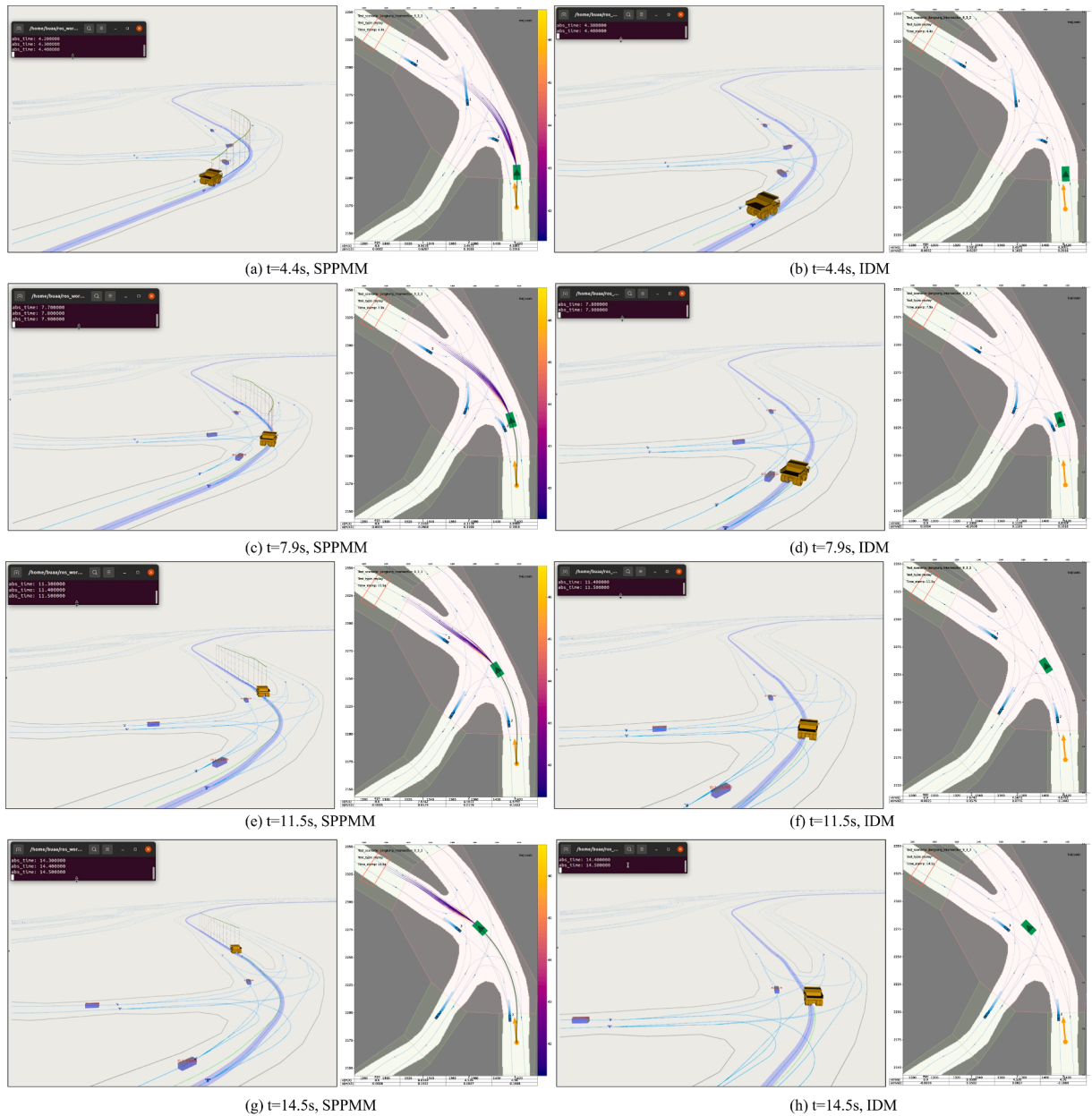


Fig. 11. Qualitative results of the two benchmarks in case “jiangxi_jiangtong_intersection_9-3-2”.

Fig. 14. For application in the mines on-road scenario, an additional obstacle avoidance decision module, Feasibility Space Analysis, is incorporated to reduce the search space.

- (1) Feasibility Space Analysis: This module determines the maximum forward road space for real-time planning algorithms in on-road scenarios, ensuring that the search or optimization path does not extend excessively. Obstacles near the road's borderline are merged into the road boundary based on their spatial relationship to reduce the introduction of excessive non-convex spaces into the Hybrid A* search module.
- (2) ESDF-based Obstacle Mapping: The Euclidean Signed Distance Field (ESDF) is highly advantageous for online motion planning in autonomous vehicles, as it enables efficient querying of obstacle distances and gradient information. Various fast, incremental ESDF map construction methods are available for real-time motion planning (Zhou et al., 2021). Our benchmark algorithm uses the ESDF map to consistently represent the distances

to road boundaries and static obstacles, enabling rapid safety distance calculations for the downstream Hybrid A* module and CiLQR-based trajectory optimization module. As illustrated in Fig. 15, the minimum obstacle distance in a local area can be queried from the ESDF map for any given input point (x_k, y_k) .

- (3) Hybrid A* Search-based Path Generation: We employed a well-established Hybrid A* Search algorithm (Dolgov et al., 2010) to generate an initial path that successfully avoids obstacles. During the node expansion process in the Hybrid A* Search, reverse driving was not considered. There are two main reasons for this: it simplifies the search space and reduces computational overhead. Additionally, the static obstacle avoidance scenario library currently provided mainly consists of wide on-road scenarios, not parking scenarios typically found in the loading and unloading areas of open-pit mines. Of course, whether omitting reverse driving affects the success rate of obstacle avoidance planning tasks in narrow on-road scenarios with static obstacles is a topic worth exploring in the future. In this benchmark, Hybrid A* is

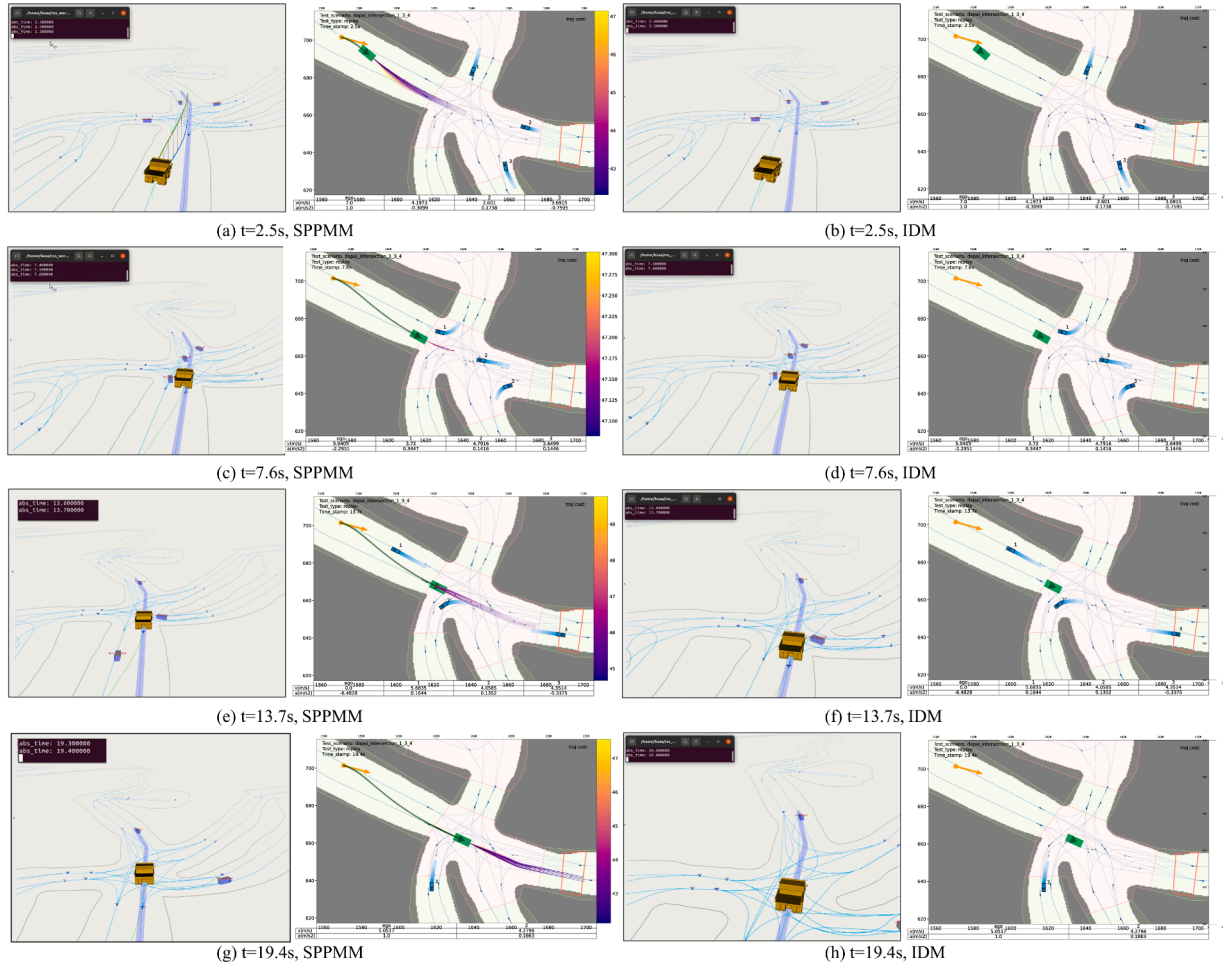


Fig. 12. Qualitative results of the two benchmarks in case “guangdong_dapai_intersection_1-3-4”.

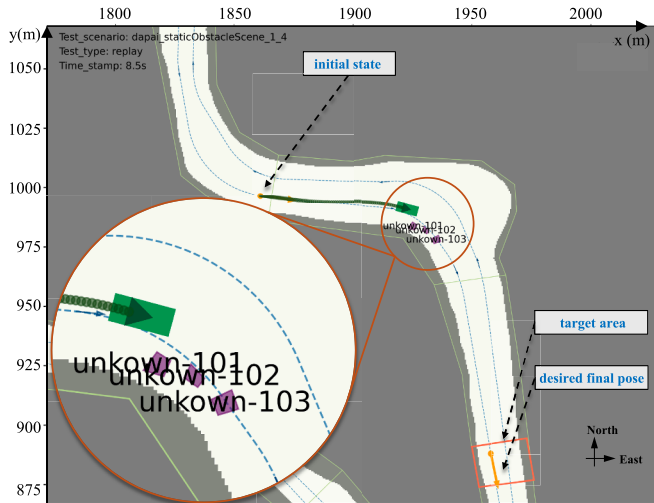


Fig. 13. Example of a static obstacle avoidance task.

employed heuristically to generate a coarse, collision-free path that guides the vehicle toward the target region.

- (4) Warm Start with Path Tracking and Constant Velocity: Prior to trajectory optimization, a warm start module was introduced to provide a reasonable initial solution to accelerate CiLQR convergence. A simple Pure-Pursuit controller (Li et al., 2023c) is employed to track the previously generated collision-free path,

and constant velocity assumption is applied to generate an initial trajectory with a 10-second planning horizon, which serves as the starting point for the CiLQR trajectory iteration.

- (5) CiLQR-based Trajectory Optimization: To achieve effective obstacle avoidance in high-curvature road scenarios in open-pit mines, a trajectory optimizer based on the CiLQR was implemented, which directly generates an optimal, coupled lateral and longitudinal trajectory. The benchmarks will be introduced in the following three sections: (A) The Basic Flow of iLQR, (B) The Principles of CiLQR, and (C) Formulating Obstacle Avoidance Trajectory Planning as a CiLQR Problem.

(A) The Basic Flow of iLQR.

The iLQR follows the Principle of Optimality and Pontryagin’s Minimum Principle. The derivation begins by defining the *unconstrained motion planning problem*, total cost function, cost-to-go function, value function, Bellman equation, and the iLQR process (Tassa et al., 2014). The basic flow of iLQR is shown in Fig. 16.

(Problem 1) The discrete-time finite-horizon *Unconstrained Motion Planning Problem* is formulated as:

$$\min_{u_1, u_2, \dots, u_{N-1}} \left\{ L_{final}(\mathbf{x}_N) + \sum_{i=k}^{N-1} L_i(\mathbf{x}_i, \mathbf{u}_i) \right\} \quad (15a)$$

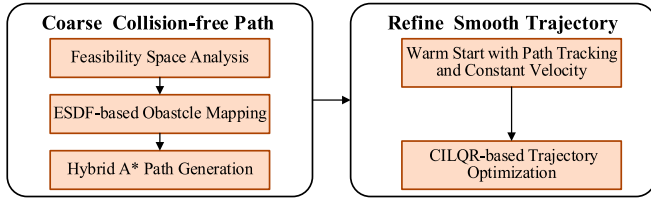
$$\text{s.t. } \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), k = 0, 1, \dots, N-1 \quad (15b)$$

$$\mathbf{x}_0 = \mathbf{x}_{start} \quad (15c)$$

Table 3

Metric design for static obstacle avoidance scenario testing.

Metric Categories	Metric Name	Metric Score Calculation Description	Points
Safety (40 points)	Collisions with Obstacles	Penalty: Safety score is 0 if a collision occurs.	0 or 40
	Collisions with Road Boundaries	Penalty: Safety score is 0 if a collision occurs.	0 or 40
	Lateral Safety (Roadside)	Penalty: Deduct points based on the duration below the lateral safety distance threshold over total driving time.	0–40
Efficiency (10 points)	Driving Duration	If the safety score is 0, this score is also 0; otherwise, score is calculated as $\min(1.0, \text{total driving time} / \text{expected time}) * 10$.	0–10
Smoothness (30 points)	Lateral	Penalty: $\min(1.0, \text{duration exceeding lateral acceleration threshold} / \text{total driving time}) * 10$.	0–10
	Longitudinal	Penalty: $\min(1.0, \text{duration exceeding longitudinal acceleration threshold} / \text{total driving time}) * 10$.	0–10
	Turning	Penalty: $\min(1.0, \text{duration exceeding centripetal acceleration threshold} / \text{total driving time}) * 10$.	0–10
Task Completion (20 points)	Progress along Route Path	If the ego vehicle reaches the goal, the score is 20; otherwise, it is calculated as $\min(1.0, \text{final driving distance along the route path} / \text{overall route path distance}) * 10$.	0–20
	Final Yaw Deviation	Penalty: If the ego vehicle does not reach the goal, score is 0; otherwise, score is $\min(1.0, \cos(\text{final yaw deviation} / 0.5 * \pi)) * 10$.	0–10

**Fig. 14.** Architecture of the benchmark HA*+CILQR planning algorithm in MineSim.

where, Eq. (15b) is the discrete-time dynamics system which describes the evolution of the state $\mathbf{x} \in \mathbb{R}^n$ from time k to $k + 1$. The generalized trajectory $\{\mathbf{X}, \mathbf{U}\}$ is defined as a sequence of states $\mathbf{X} \triangleq [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$ and corresponding controls sequence $\mathbf{U} \triangleq [\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{N-1}]$ satisfying the dynamics system.

Total cost function: The total cost function J is denoted by the sum of running costs J^i and final cost J^{final} , incurred when starting from $\mathbf{x}_0$ and applying $\mathbf{U}$ until the horizon N is reached:

$$J(\mathbf{x}_0, \mathbf{U}) = L_{\text{final}}(\mathbf{x}_N) + \sum_{i=0}^{N-1} L_i(\mathbf{x}_i, \mathbf{u}_i) \quad (16)$$

The solution of the optimal control problem is the minimizing control sequence:

$$\mathbf{U}^* \triangleq \underset{\mathbf{U}}{\text{argmin}}(J(\mathbf{x}_0, \mathbf{U})) \quad (17)$$

Cost-to-go function: the cost-to-go function J^k is the partial sum of costs from k to N , expressed as:

$$J_k(\mathbf{x}_k, \mathbf{U}_k) = L_{\text{final}}(\mathbf{x}_N) + \sum_{i=0}^{N-1} L_i(\mathbf{x}_i, \mathbf{u}_i) \quad (18a)$$

$$\mathbf{U}_k \triangleq [\mathbf{u}_k, \dots, \mathbf{u}_{N-1}] \quad (18b)$$

Value function: The value function V_k is defined as the optimal cost-to-go function, representing the minimum cost from a given state $\mathbf{x}_k$ to the final state:

$$V_k(\mathbf{x}_k) \triangleq \min_{\mathbf{U}_k} [J_k(\mathbf{x}_k, \mathbf{U}_k)] \quad (19)$$

Bellman equation: The theoretical foundation of iLQR is Dynamic Programming, specifically, the following Bellman equation:

$$\begin{aligned} V_k(\mathbf{x}_k) &= \min_{\mathbf{U}_k} [L_k(\mathbf{x}_k, \mathbf{u}_k) + V_{k+1}(\mathbf{x}_{k+1})] \\ &= \min_{\mathbf{U}_k} [L_k(\mathbf{x}_k, \mathbf{u}_k) + V_{k+1}(f(\mathbf{x}_k, \mathbf{u}_k))] \end{aligned} \quad (20)$$

The value function with the terminal cost is defined as $V_N(\mathbf{x}) \triangleq L_N(\mathbf{x}_N)$.

Initialize (step 1 of iLQR): Set the initial state, the initial control sequence $\tilde{\mathbf{U}} = [\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{N-1}]$, and initial regularization parameter $\lambda_0 = 1.0$. As shown in Fig. 16, for the trajectory planning problem, a reference trajectory is also initialized for calculating initial cost.

Forward pass (step 2 of iLQR): The forward pass consists of two parts. The first part involves simulating system dynamics as described in Eq. (15b) to generate the prior trajectory $\{\tilde{\mathbf{X}}, \tilde{\mathbf{U}}\}$, from which the overall

cost $J_{\text{old}}(\mathbf{x}_0, \tilde{\mathbf{U}})$ of the prior trajectory can be calculated. The second part involves calculating the various first and second derivatives of the system: $\mathbf{F}_x^k, \mathbf{F}_u^k, \mathbf{L}_x^k, \mathbf{L}_u^k, \mathbf{L}_{xx}^k, \mathbf{L}_{ux}^k, \mathbf{L}_{uu}^k$. These refer to the first derivatives of $f(\mathbf{x}_k, \mathbf{u}_k)$ with respect to $\mathbf{x}_k$ and $\mathbf{u}_k$. Additionally, it needs the first and second derivatives of $L_{\text{final}}(\mathbf{x}_N)$ and $L_k(\mathbf{x}_k, \mathbf{u}_k)$ with respect to $\mathbf{x}_k$ and $\mathbf{u}_k$.

Backward pass (step 3 of iLQR): Once the initial trajectory $\{\tilde{\mathbf{X}}, \tilde{\mathbf{U}}\}$ is obtained, the backward pass is used to drive the value function, as described in Eq. (20), toward a local minimum. This process computes a local solution by applying a quadratic Taylor expansion. Then, define $Q_k(\delta \mathbf{x}_k, \delta \mathbf{u}_k)$ to be the change in the value function at $(\mathbf{x}_k, \mathbf{u}_k)$ as a result of small perturbations $(\delta \mathbf{x}_k, \delta \mathbf{u}_k)$:

$$\begin{aligned} Q_k(\delta \mathbf{x}_k, \delta \mathbf{u}_k) &= L_k(\tilde{\mathbf{x}}_k + \delta \mathbf{x}_k, \tilde{\mathbf{u}}_k + \delta \mathbf{u}_k) - L_k(\tilde{\mathbf{x}}_k, \tilde{\mathbf{u}}_k) + V_{k+1}(f(\tilde{\mathbf{x}}_k \\ &\quad + \delta \mathbf{x}_k, \tilde{\mathbf{u}}_k + \delta \mathbf{u}_k)) - V_{k+1}(f(\tilde{\mathbf{x}}_k, \tilde{\mathbf{u}}_k)) \end{aligned} \quad (21)$$

Approximate $Q_k(\delta \mathbf{x}_k, \delta \mathbf{u}_k)$ using its second-order Taylor expansion to obtain:

$$Q_k(\delta \mathbf{x}_k, \delta \mathbf{u}_k) \approx \frac{1}{2} \begin{bmatrix} 1 \\ \delta \mathbf{x}_k \\ \delta \mathbf{u}_k \end{bmatrix}^T \begin{bmatrix} 0 & (\mathbf{Q}_x^k)^T & (\mathbf{Q}_u^k)^T \\ \mathbf{Q}_x^k & \mathbf{Q}_{xx}^k & \mathbf{Q}_{xu}^k \\ \mathbf{Q}_u^k & \mathbf{Q}_{ux}^k & \mathbf{Q}_{uu}^k \end{bmatrix} \begin{bmatrix} 1 \\ \delta \mathbf{x}_k \\ \delta \mathbf{u}_k \end{bmatrix} \quad (22a)$$

$$\mathbf{Q}_x^k = \mathbf{L}_x^k + (\mathbf{F}_x^k)^T \mathbf{V}_x^{k+1} \quad (22b)$$

$$\mathbf{Q}_u^k = \mathbf{L}_u^k + (\mathbf{F}_u^k)^T \mathbf{V}_u^{k+1} \quad (22c)$$

$$\mathbf{Q}_{xx}^k = \mathbf{L}_{xx}^k + (\mathbf{F}_x^k)^T \mathbf{V}_{xx}^{k+1} \mathbf{F}_x^k + \mathbf{V}_x^{k+1} \mathbf{F}_{xx}^k \quad (22d)$$

$$\mathbf{Q}_{ux}^k = \mathbf{Q}_{xu}^k = \mathbf{L}_{ux}^k + (\mathbf{F}_x^k)^T \mathbf{V}_{xx}^{k+1} \mathbf{F}_u^k + \mathbf{V}_x^{k+1} \mathbf{F}_{ux}^k \quad (22e)$$

$$\mathbf{Q}_{uu}^k = \mathbf{L}_{uu}^k + (\mathbf{F}_u^k)^T \mathbf{V}_{xx}^{k+1} \mathbf{F}_u^k + \mathbf{V}_u^{k+1} \mathbf{F}_{uu}^k \quad (22f)$$

Optimal control modification: The optimal control modification $\delta \mathbf{u}_k^*$ for some state perturbation $\delta \mathbf{u}_k$ is obtained by minimizing the quadratic model as described in Eq. (22a):

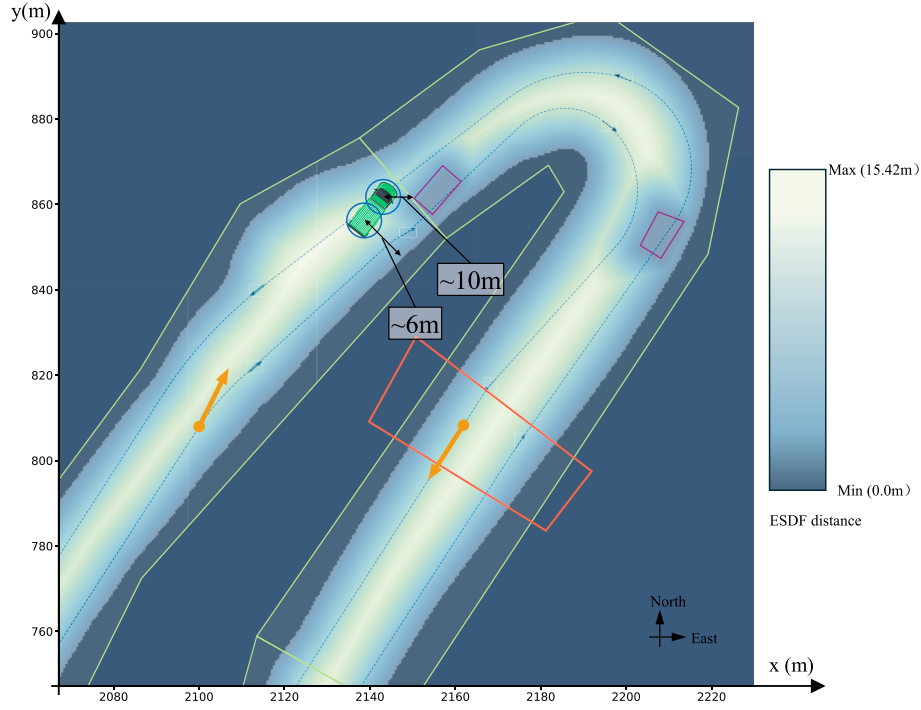


Fig. 15. The diagram of querying the minimum obstacle distance based on the ESDF map.

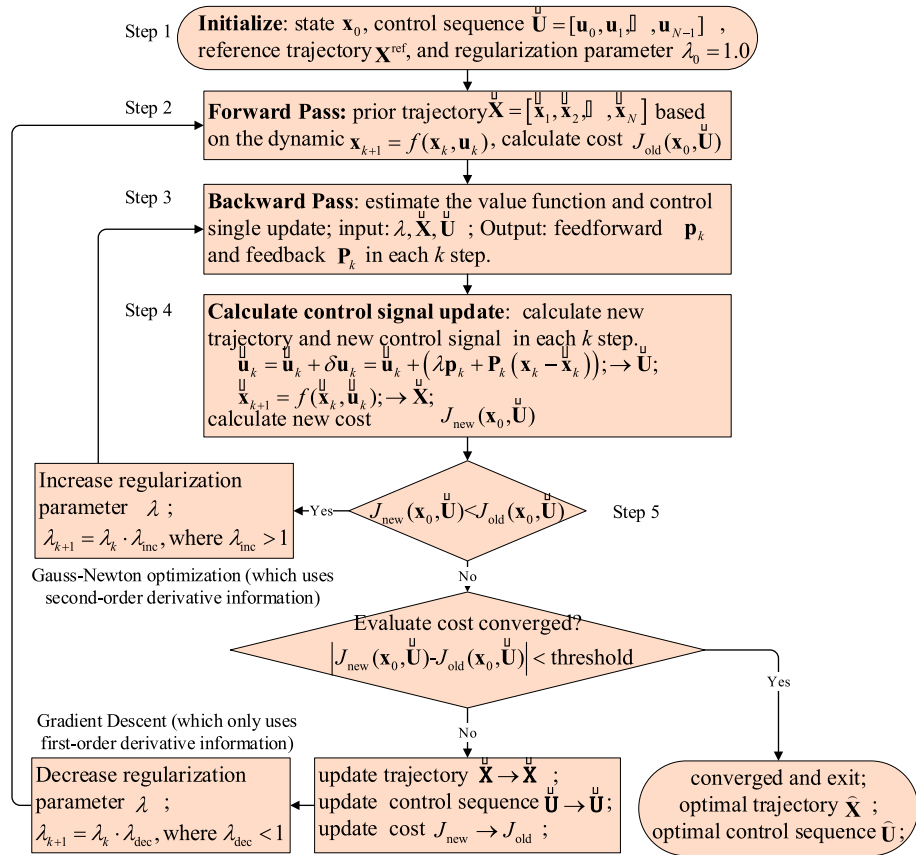


Fig. 16. The basic flow of iLQR-based trajectory optimization.

$$\begin{aligned} \delta \mathbf{u}_k^* &= \min_{\delta \mathbf{u}_k} Q_k(\delta \mathbf{x}_k, \delta \mathbf{u}_k) = \mathbf{p}_k + \mathbf{P}_k \delta \mathbf{x}_k \\ &= \left[-(\mathbf{Q}_{uu}^k + \lambda \mathbf{I})^{-1} \mathbf{Q}_{ux}^k \right] + \left[-(\mathbf{Q}_{uu}^k + \lambda \mathbf{I})^{-1} \mathbf{Q}_{ux}^k \right] \delta \mathbf{x}_k \end{aligned} \quad (23)$$

where $\mathbf{p}_k$ is the feedforward term for the control signal update, $\mathbf{P}_k \delta \mathbf{x}_k$ is feedback term. $\lambda \mathbf{I}$ is a regularization term (diagonal matrix with regularization parameter λ added to $\mathbf{Q}_{uu}^k$ to ensure positive definiteness. By computing the partial derivatives Q_k at each time step using backward recursion (from N to 0), the derivatives and Hessians of the value function $V_k(\mathbf{x}_k)$ with respect to the state vector $\mathbf{x}$ are as:

$$\mathbf{v}_x^k = \mathbf{Q}_x^k - \mathbf{Q}_u^k (\mathbf{Q}_{uu}^k)^{-1} \mathbf{Q}_{ux}^k \quad (24a)$$

$$\mathbf{v}_{xx}^k = \mathbf{Q}_{xx}^k - \mathbf{Q}_{xu}^k (\mathbf{Q}_{uu}^k)^{-1} \mathbf{Q}_{ux}^k \quad (24b)$$

Calculate control signal update (step 4 of iLQR): After the backward pass, forward simulate the new trajectory using the system dynamics in Eq. (15b) and update the control signal:

$$\hat{\mathbf{x}}_0 = \mathbf{x}_{start} \quad (25a)$$

$$\hat{\mathbf{u}}_k = \tilde{\mathbf{u}}_k + \delta \mathbf{u}_k = \tilde{\mathbf{u}}_k + (\lambda \mathbf{p}_k + \mathbf{P}_k \delta \mathbf{x}_k) = \tilde{\mathbf{u}}_k + (\lambda \mathbf{p}_k + \mathbf{P}_k (\hat{\mathbf{x}}_k - \tilde{\mathbf{x}}_k)) \quad (25b)$$

$$\hat{\mathbf{x}}_{k+1} = f(\hat{\mathbf{x}}_k, \hat{\mathbf{u}}_k) \quad (25c)$$

Upon completing one iteration from Step 1 to Step 4, the prior trajectory $\{\tilde{\mathbf{x}}, \tilde{\mathbf{u}}\}$ is replaced by the posterior new trajectory $\{\hat{\mathbf{x}}, \hat{\mathbf{u}}\}$, resulting in improved performance.

Evaluate converge (step 4 of iLQR): The convergence of the iteration is assessed using the Levenberg-Marquardt algorithm (Levenberg, 1944). If the updates are progressing well (the cost of the new trajectory is lower than that of the previous one), gradient information is included to accelerate trajectory optimization (repeat step 2). If the updates are unsatisfactory, gradient descent is scaled back (repeat step 3), relying on first-order derivative information and using smaller step sizes. In step 3, λ is a backtracking search parameter, initially set to 1.0 and iteratively reduced. Finally, this backward-forward process is repeated until convergence to the locally optimal trajectory.

(B) The Principles of CiLQR.

Although iLQR is a highly efficient algorithm for solving motion planning problems with nonlinear system dynamics, its major drawback lies in its inability to handle inequality constraints, which are crucial in autonomous driving motion planning scenarios. Inequality constraints, such as those for collision avoidance and actuator limits, are typical in these scenarios. The following sections will introduce methods for handling the inequality-constrained iLQR problem as:

$$\min_{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_{N-1}} \left\{ L_{final}(\mathbf{x}_N) + \sum_{i=k}^{N-1} L_i(\mathbf{x}_i, \mathbf{u}_i) \right\} \quad (26a)$$

$$\text{s.t. } \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), k = 0, 1, \dots, N-1 \quad (26b)$$

$$\mathbf{x}_0 = \mathbf{x}_{start} \quad (26c)$$

$$g_x^k(\mathbf{x}_k, \mathbf{u}_k) < 0, k = 1, 2, \dots, N \quad (26d)$$

$$g_u^k(\mathbf{x}_k, \mathbf{u}_k) < 0, k = 0, 1, \dots, N-1 \quad (26e)$$

A feasible way to handle constraints is by introducing penalties. The basic idea is to use a barrier function to reshape the constraint functions in Eq. (26d) and Eq. (26e):

$$c(\mathbf{x}_k, \mathbf{u}_k) = b(g^k(\mathbf{x}_k, \mathbf{u}_k)) \quad (27)$$

Then, the barrier function is added to the objective function. The

ideal function is the indicator function:

$$b(g(\mathbf{x}_k, \mathbf{u}_k)) = q_1 \exp(q_2 g(\mathbf{x}_k, \mathbf{u}_k)) \quad (28)$$

where q_1 and q_2 are parameters to tune the shape of the barrier function. For the barrier function to be integrated into the solution process of the iLQR algorithm, it must be quadratically linearized. The Jacobian and Hessian matrices of the barrier function are shown below:

$$b_x = q_1 q_2 \exp(q_2 g(\mathbf{x}_k, \mathbf{u}_k)) g_x \quad (29a)$$

$$b_u = q_1 q_2 \exp(q_2 g(\mathbf{x}_k, \mathbf{u}_k)) g_u \quad (29b)$$

$$b_{xx} = q_1 q_2 \exp(q_2 g(\mathbf{x}_k, \mathbf{u}_k)) (q_2 g_x g_x^T + g_{xx}) \quad (29c)$$

$$b_{xu} = q_1 q_2 \exp(q_2 g(\mathbf{x}_k, \mathbf{u}_k)) (q_2 g_x g_u^T + g_{xu}) \quad (29d)$$

$$b_{uu} = q_1 q_2 \exp(q_2 g(\mathbf{x}_k, \mathbf{u}_k)) (q_2 g_u g_u^T + g_{uu}) \quad (29e)$$

(C) Formulating Obstacle Avoidance Trajectory Planning as a CiLQR Problem.

If the CiLQR algorithm is applied to the on-road autonomous driving motion planning problem, it is necessary to clarify the physical meaning of several key equations from Eq. (15) to Eq. (29).

First, the nonlinear system dynamics in Eq. (26b) are modeled as:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \quad (30a)$$

$$f_1 : \mathbf{x}_{k+1} = v_k \cdot \cos(\theta_k) \cdot T \quad (30b)$$

$$f_2 : y_{k+1} = v_k \cdot \sin(\theta_k) \cdot T \quad (30c)$$

$$f_3 : \theta_{k+1} = \theta_k + \frac{v_k \cdot \tan(\delta_k)}{L_{wb}} \cdot T \quad (30d)$$

$$f_4 : v_{k+1} = v_k + a_k \cdot T \quad (30e)$$

where the states of the ego vehicle are defined as $\mathbf{x}_k = [x_k, y_k, \theta_k, v_k]^T$, and the control input vector $\mathbf{u}_k = [a_k, \delta_k]^T$ includes acceleration and steering angle. Note that this vehicle bicycle kinematic model differs slightly from Eq. (2a)–(2e).

Second, the cost function in Eq. (16) can be decomposed into control costs, reference offset costs, and constraint costs:

$$J = \sum_{k=0}^{N-1} (C_k^u + C_k^{\text{offset}} + C_k^{\text{constraint}}) \quad (31)$$

The *control costs* include penalty terms for acceleration and steering angle as:

$$C_k^u = C_k^{\text{acc}} + C_k^{\delta} = \mathbf{u}_k^T \begin{bmatrix} w_{\text{acc}} & 0 \\ 0 & w_{\delta} \end{bmatrix} \mathbf{u}_k \quad (32)$$

The *reference offset costs* penalize the distance from the reference path and the deviation from the reference speed as:

$$\begin{aligned} \Delta \mathbf{x}_k &= [\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}, y_k - y_k^{\text{ref}}, 0, v_k - v_k^{\text{ref}}] \\ C_k^{\text{offset}} &= C_k^{\text{pose}} + C_k^v = \Delta \mathbf{x}_k^T \text{diag}\{w_{\text{pose}}, w_{\text{pose}}, 0, w_v\} \Delta \mathbf{x}_k \end{aligned} \quad (33)$$

Constraint costs include the *acceleration constraint*, *steering angle constraint*, *boundary constraint*, and *obstacle avoidance constraint*, all of which are inequality constraints. These can be integrated into the cost function using Eqs. (27)–(29e). These terms are as follows.

The acceleration constraint and steering angle constraint reflect the limits of the powertrain and steering mechanisms, as shown below:

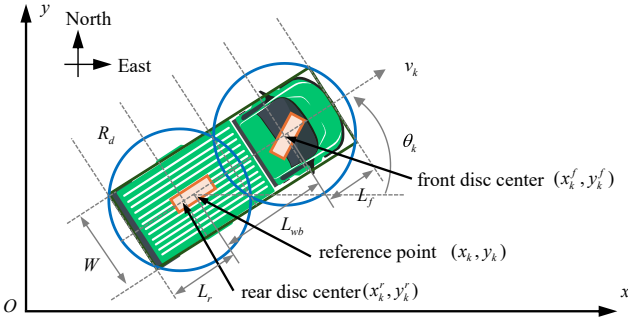


Fig. 17. Schematic diagram of two discs used to evenly cover the vehicle body.

Table 4

Quantitative experimental score results for static obstacle avoidance scenario testing.

Algorithms	Safety	Efficiency	Smoothness	Task completion	Total Score
SPPMM	36.5	9.1	25.6	18.2	89.4
HA*-CiLQR	39.2	9.0	28.2	19.5	95.9

$$C_k^{cons,acc} : a_{min} \leq u_k^T \begin{bmatrix} 1 \\ 0 \end{bmatrix} \leq a_{max} \quad (34)$$

$$C_k^{cons,\delta} : -\delta_{max} \leq u_k^T \begin{bmatrix} 0 \\ 1 \end{bmatrix} \leq \delta_{max}$$

The boundary and obstacle avoidance constraints are described using the minimum obstacle distance queried from an ESDF-based obstacle map. The unified constraint is as:

$$d(x_k) = f_{ESDF,min}(x_k) - d_{safe} - R_D \geq 0 \quad (35)$$

where d_{safe} represents the safe distance the ego vehicle needs to maintain from the boundary and other obstacles. To simplify the

distance relationship between the rectangular vehicle and obstacles, two discs are used to cover the vehicle body evenly (Fig. 17). The disc centers, namely (x_k^f, y_k^f) and (x_k^r, y_k^r) , are quartile points along the vehicle's longitudinal axis as defined in Eq. (36a)–(36e). R_D is the radius of either disc as defined in Eq. (36e). $f_{ESDF,min}(x_k)$ represents the ego vehicle's rear axle center pose, which is used as input and transformed into the two-disc centers (x_k, y_k, θ_k) , from which the minimum obstacle distance is queried via the ESDF obstacle map.

$$x_k^f = x_k + 1/4(3L_{wb} + 3L_f - L_r)\cos\theta_k \quad (36a)$$

$$y_k^f = y_k + 1/4(3L_{wb} + 3L_f - L_r)\sin\theta_k \quad (36b)$$

$$x_k^r = x_k + 1/4(L_{wb} + L_f - 3L_r)\cos\theta_k \quad (36c)$$

$$y_k^r = y_k + 1/4(L_{wb} + L_f - 3L_r)\sin\theta_k \quad (36d)$$

$$R_D = 1/2 \sqrt{(0.5(L_r + L_{wb} + L_f))^2 + W^2} \quad (36e)$$

5.3.2. Test results of static obstacle avoidance benchmark

- (1) Experiment Setup: Two benchmark algorithms were evaluated for the static obstacle avoidance task, automatically tested across 22 scenarios. The simulation engine was configured to *Static Replay Test Mode* (one can refer to Appendix C for all configurable methods). The ego closed-loop simulation was configured with the LQR-based motion controller and KBM-wRL ego update model.
- (2) Quantitative Analysis: Utilizing the metrics designed in Table 3, the proposed benchmarks were comprehensively evaluated with the quantitative experimental results presented in Table 4. Overall, the HA*-CiLQR method demonstrates superior performance; however, this algorithm prioritizes safety metrics by maintaining a distance from obstacles, resulting in slightly lower efficiency than the SPPMM method. It is important to note that we only provided the implementations of the two benchmarks while many related parameters have not been optimized yet.

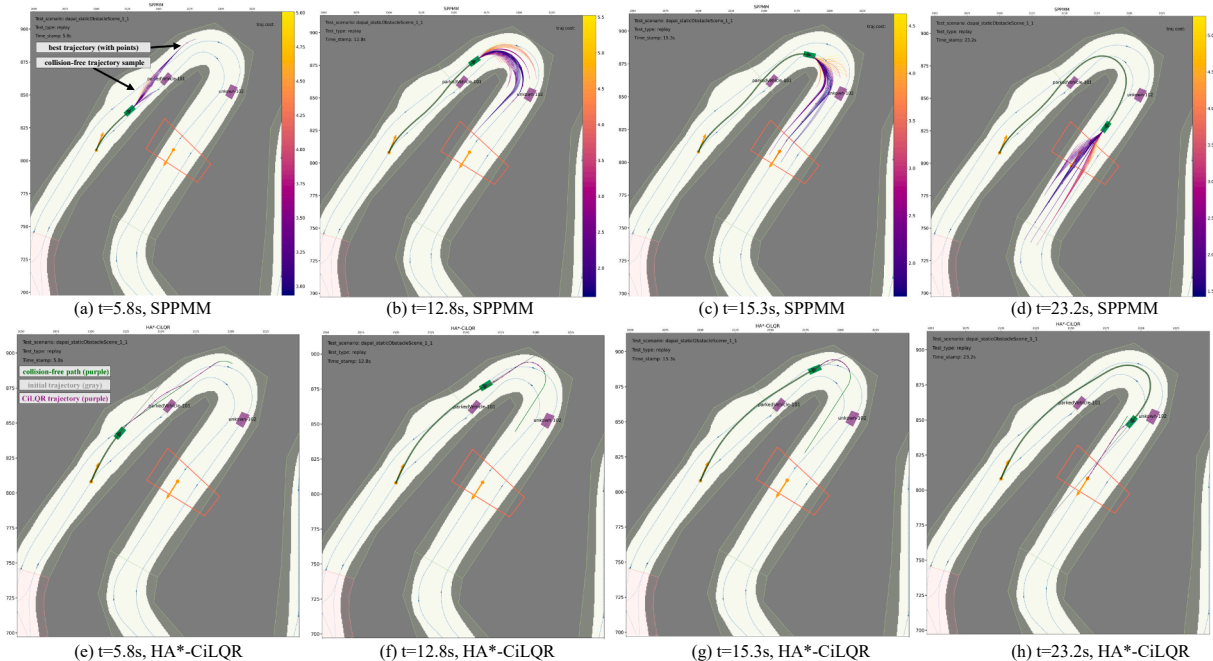


Fig. 18. Qualitative results of the two benchmark benchmarks in case "dapai_staticObstacleScene_1-1".

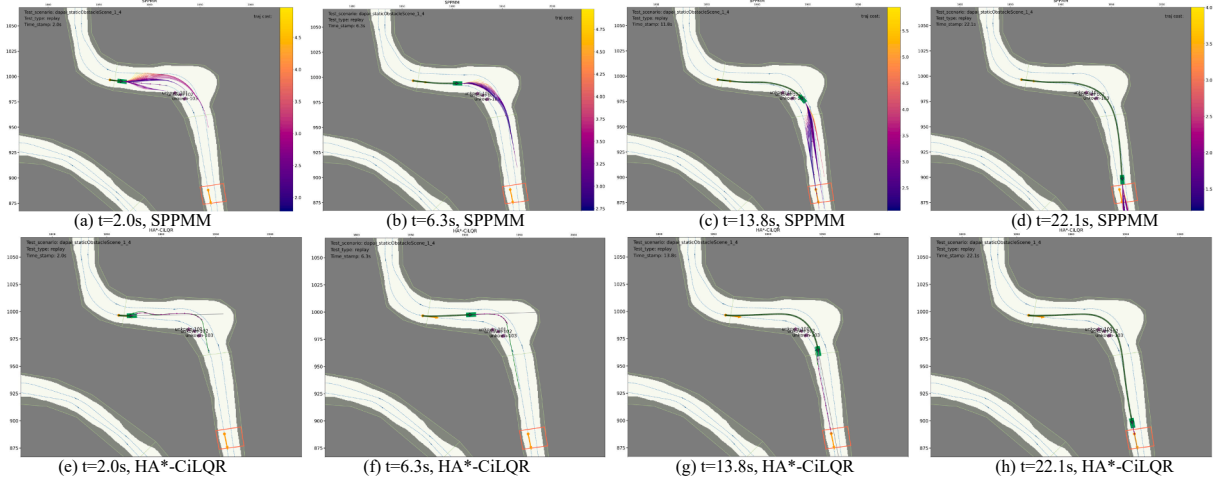


Fig. 19. Qualitative results of the two benchmarks in case “dapai_staticObstacleScene_1-4”.

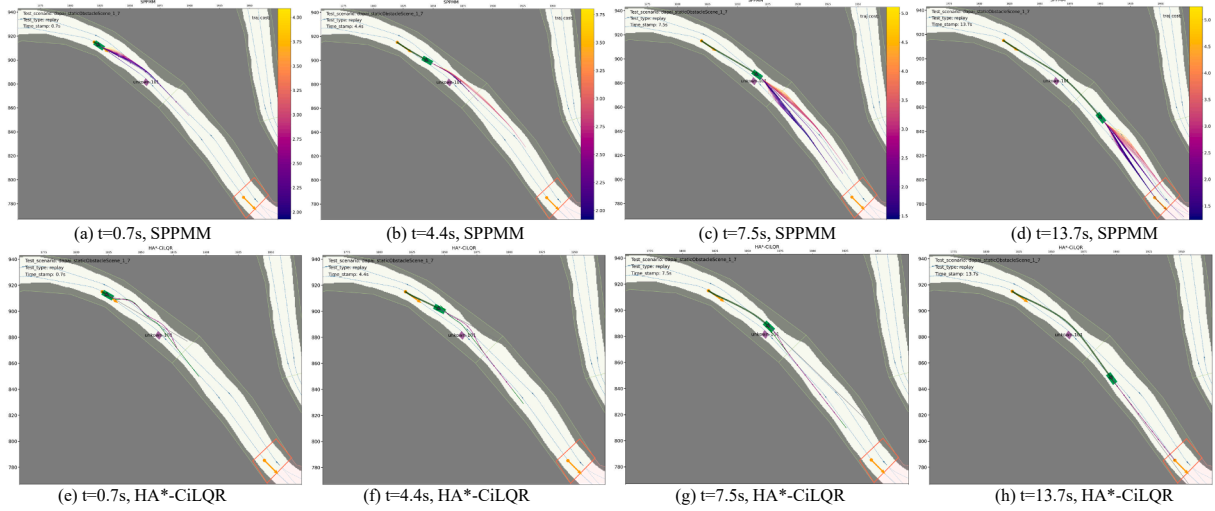


Fig. 20. Qualitative results of the two benchmarks in case “dapai_staticObstacleScene_1-7”.

- (3) **Qualitative Analysis:** The 2D visualizer was utilized to illustrate the obstacle avoidance processes in three tested scenarios (see Fig. 18, Fig. 19 and Fig. 20). As shown in Fig. 18(a), collision-free trajectory samples are represented by multiple-colored lines, indicating trajectories that have passed collision checks and dynamics constraints. The “best trajectory with points” represents the optimal path determined through cost evaluation. Additionally, Fig. 18(a) highlights that the smoothness of the reference path influences the trajectories sampled in the Frenet frame; if the reference path quality is poor, the resulting sampled trajectories do not align with vehicle kinematics. In Fig. 18(e), the green line represents a collision-free path based on the hybrid A* search that can reach the target area. In contrast, the gray line illustrates the initial trajectory from CiLQR, and the purple line the trajectory optimized through iterative refinement with CiLQR. As shown in Fig. 20(e), the optimized trajectory achieves the safest path between obstacles and the road boundaries.

6. Conclusions and future work

This study introduces MineSim, an open-source simulation system designed specifically for planning tasks of autonomous trucks in open-pit mines. MineSim provides scenario-based closed-loop testing

functionality, evaluation metrics system, and 2D/3D visualization tools. It processes real-world traffic scene data from open-pit mines and offers two scenario libraries for reproducible research. First, the dynamic obstacle avoidance scenario library focuses on planning algorithms for unstructured intersection conflict scenarios involving other vehicles. Two benchmark methods were provided: the Intelligent Driver Model (IDM) and the Sampling Planner using Predefined Maneuver Modes (SPPMM). Second, the static obstacle avoidance scenario library focuses on planning algorithms for handling static obstacles commonly encountered by large mining trucks in unstructured road environments. Similarly, two benchmark methods were offered for the static obstacle avoidance test: SPPMM and an approach combining Hybrid A* with the Constrained Iterative Linear Quadratic Regulator (CiLQR). The detailed quantitative tests and qualitative experimental analyses of these methods were provided. The experimental results verify that MineSim offers a reproducible testing tool and scenario data for autonomous driving planning tasks in open-pit mines, with the potential to serve as a valuable resource for advancing autonomous driving technologies in non-urban and unstructured road environments.

However, it should be noted that MineSim has several limitations compared to current scenario-based testing tools for urban autonomous driving. First, MineSim’s scenario data is relatively limited, currently consisting of over 300 scenarios from real-world data (“dataset-

dynamicsscenario-v1.0" includes 210 intersection scenarios and "dataset-staticscenario-v1.0" includes 22 static obstacle scenarios). This dataset size is insufficient to support data-driven methods such as modern deep learning and imitation learning. Additionally, enhancing real-time mining traffic scene by rendering and introducing more robust scenario generation models to simulate extreme situations would provide further reproducible experimental material, facilitating more profound research into autonomous driving technologies for unstructured road environments.

CRedit authorship contribution statement

Zhifa Chen: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Guizhen Yu:** Writing – review & editing, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Peng Chen:** Writing – review & editing, Validation, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Guoxi Cao:** Writing – review & editing, Visualization, Validation, Supervision, Software, Methodology, Data curation, Conceptualization. **Zheng Li:** Writing – review & editing, Visualization, Validation, Software, Formal

analysis, Data curation, Conceptualization. **Yifang Zhang:** Visualization, Validation, Supervision, Software, Resources, Methodology, Investigation, Formal analysis, Data curation. **Haoyuan Ni:** Writing – review & editing, Visualization, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation. **Bin Zhou:** Writing – review & editing, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Jian Sun:** Writing – review & editing, Project administration, Methodology, Formal analysis, Data curation. **Huanyu Ban:** Writing – review & editing, Visualization, Software, Resources, Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work is supported by the National Natural Science Foundation of China (No. 52272327) and the National Key Technologies R&D Program of China (No. 2022YFB4703700).

Appendix A:. Description of the mines HD map format

The HD map consists of two components: a raster map and a semantic map.

- (1) Raster Map: The high-resolution mask map data is used to represent the drivable areas of open-pit mining roads. This format preserves the maximum detail for all drivable areas within the entire area, as illustrated in Fig. 3 (c) and (f). In the raster map, each pixel contains a value of either 0 or 1, indicating non-drivable and drivable areas, respectively. The pixel coordinates are precisely mapped to absolute geographical coordinates, with a scale of 10 pixels per meter.
- (2) Semantic Map: The high-precision semantic map consists of geometric and semantic layers within the entire area, with precise location data for each layer (see Fig. 22). The geometric layers include "node," "node_block," and "polygon," recording various geometric elements like points and surfaces that compose the semantic map. The semantic layers represented by geometric polygons include "road," "intersection," "loading_area," "unloading_area," and "road_block." These layers divide drivable areas of the entire mines into smaller regions such as road segments, intersections, loading areas, and unloading areas, each associated with different potential traffic conflicts. The semantic layers represented by geometric lines include "dubins_pose," "reference_path," and "borderline." The geometric and semantic layers are managed by a unique identifier called a "token", ensuring the uniqueness of each element within the map system. The meanings of the fields and additional details are as follows:
 - "node" or "node_block": A geometric layer that records geometric point elements composing various semantic layers on the map.
 - "polygon": A geometric layer that records the geometric polygon elements that form various semantic layers on the map. It also stores association information with other semantic layers such as "reference_path," "borderline," and "dubins_pose."
 - "road": A semantic layer that records the semantic segmentation of all road segments in a specific area.
 - "intersection": A semantic layer that records the semantic segmentation of all intersections (or junctions) in a specific area.
 - "loading_area": A semantic layer that records the semantic segmentation of all loading areas (used for loading ore onto mining trucks) in a specific area.
 - "unloading_area": A semantic layer that records the semantic segmentation of all unloading areas (used for unloading ore from mining trucks) in a specific area.
 - "road_block": A semantic layer that records the semantic segmentation of all road block areas in a specific region.
 - "dubins_pose": A semantic layer used as control points for generating reference paths. It includes fields such as "dubinspose_type" and "link_polygon_token." The "dubinspose_type" records the control point types (as shown in Fig. 21a): 1) "normal": A standard control point with one upstream and one downstream path; 2) "split": One upstream path and multiple downstream paths; 3) "merge": Multiple upstream paths and one downstream path.
 - "reference_path": A semantic layer that records reference path segments, generated using Dubins curves from dubins_pose points. It includes the following fields: related polygons ("link_polygon_tokens"), related dubins_pose ("link_dubinspose_tokens"), upstream paths ("incoming_tokens"), downstream paths ("outgoing_tokens"), whether the start of the path is blocked ("is_start_blocked"), and whether the endpoint is blocked ("is_end_blocked"). Additionally, it contains "waypoints" with details such as [x coordinate, y coordinate, yaw, elevation, slope], discretely sampled at approximately 0.2-meter intervals. There exist two types of reference paths (as shown in Fig. 21.a): 1) "connector_path" represents internal paths within intersections connecting road segments; 2) "base_path" represents the basic path within a road, similar to the centerline of a two-way lane in urban roads.
 - "borderline": A semantic layer that records road boundary information. The boundary line is generated from offline data collected by mapping vehicles and corresponds to the black-and-white boundary of the raster map (bitmap_mask.png). This layer is associated with the polygon specified by the token using "link_polygon_token." Additionally, it contains "borderpoints" with details such as [x coordinate, y coordinate,

elevation], discretely sampled at approximately 0.5-meter intervals. The borderline layer has two types (as shown in Fig. 21 b): 1) “edge” represents the outer boundaries of the road, usually non-closed; 2) “inner” represents internal dividers such as walls or roundabouts, usually closed within the road.

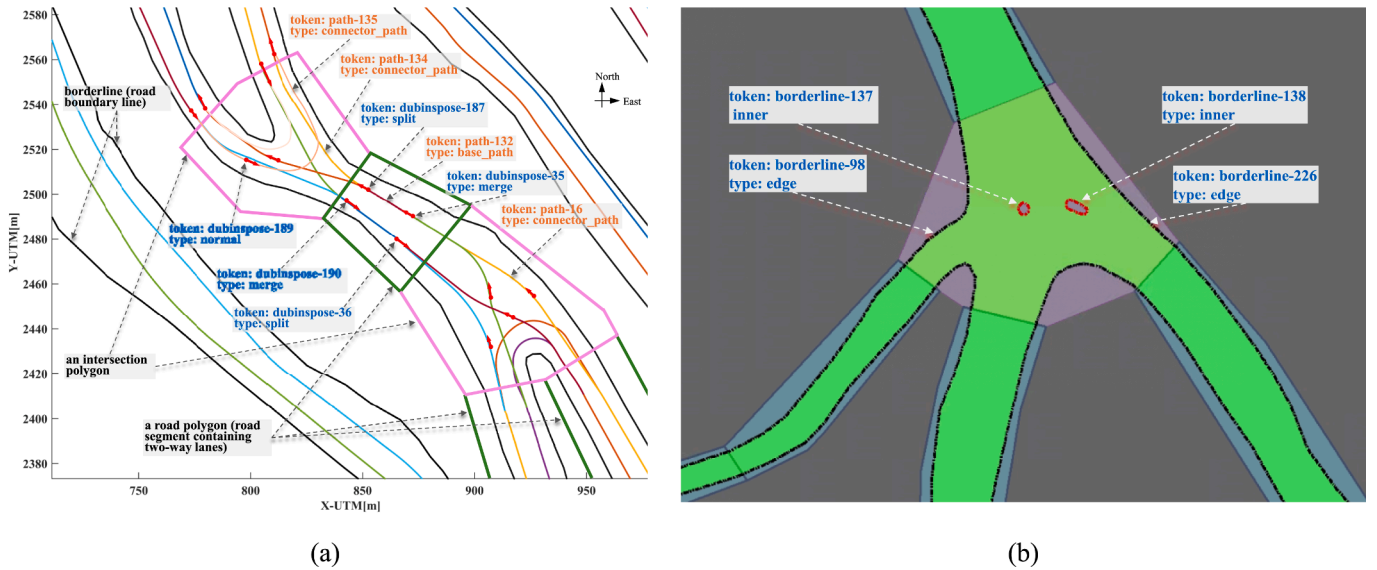


Fig. 21. Differences between types in the “dubins_pose,” “reference_path,” and “borderline” layer.

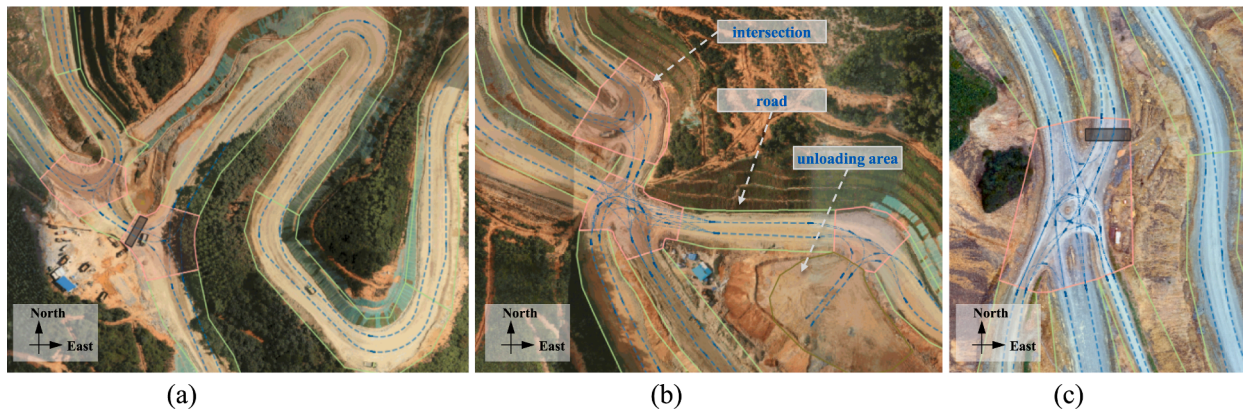


Fig. 22. Illustration of the different semantic layers in MineSim's HD map as represented in the real world.

Appendix B: Mining vehicle parameters

In MineSim system, two common types of mining trucks used in open-pit mines are provided, as shown in Fig. 23 and Fig. 24.

- A small-capacity mining truck (rated for 90 tons), a wide-body dump trucks produced by Xuzhou Construction Machinery Group (vehicle ID in MineSim: XG90G).
- A large-capacity mining truck (rated for 200 tons), an electric drive mining truck produced by Inner Mongolia North Hauler Joint Stock Co., Ltd. (vehicle ID in MineSim: NTE200).



Fig. 23. The mines truck XG90G in MineSim.



Fig. 24. The mines truck NTE200 in MineSim.

Additionally, the detailed vehicle parameters configured in MineSim code are provided below [Table 5](#) and [Table 6](#).

Table 5
Model parameters for mining trucks.

Parameter Name	Symbol	Unit	Value of XG90G	Value of NTE200
Vehicle length	L	m	9.0	13.0
Vehicle width	W	m	4.0	6.7
Vehicle wheelbase	L_{wb}	m	5.3	9.6
Distance from the reference point (rear wheel) to the front of the vehicle	L_f	m	6.5	9.2
Distance from the reference point (rear wheel) to the rear of the vehicle	L_r	m	2.5	3.8

Table 6
Constraint parameters for mining trucks (empty-load condition).

Parameter Name	Symbol	Unit	Value of MT90G	Value of MT200
Minimum steering angle	$\delta_{\min}$	rad	-1.8	-1.5
Maximum steering angle	$\delta_{\max}$	rad	1.8	1.5
Minimum steering velocity	$\omega_{\delta,\min}$	rad/s	0.3	0.2
Maximum steering velocity	$\omega_{\delta,\max}$	rad/s	-0.3	-0.2
Minimum longitudinal velocity	$v_{\min}$	m/s	0	0
Maximum longitudinal velocity	$v_{\max}$	m/s	16.7 (60 Km/h)	12.5 (45 Km/h)
Minimum longitudinal acceleration	$a_{lon,\min}$	m/s^2	-15.5	-8.9
Maximum longitudinal acceleration	$a_{lon,\max}$	m/s^2	16.8	10.2
Maximum centripetal acceleration	$a_{cen,\max}$	m/s^2	1.5	1.5

Appendix C:. Key component support Explanation in MineSim

MineSim is primarily designed for testing autonomous driving planning tasks. Specifically, it currently includes two defined planning tasks: dynamic obstacle avoidance planning and static obstacle avoidance planning. However, other components are also crucial for comprehensive autonomous driving testing, such as prediction algorithms for dynamic obstacle vehicles and the motion controller for the ego vehicle. Additionally, the Metrics Evaluation and Scenario Visualization components are designed to be customizable, allowing for the modification. Table 7 lists the components in MineSim that support user modification and extension.

Table 7

Components in MineSim that support user modification and extension.

Component		Supporting Parameter Modification	Supporting Methods Extension	Explanation
Simulation Engine	Prediction Aalgorithms	Yes	Yes	Supporting extension of advanced learning-based prediction algorithms.
	Planning Aalgorithms	Yes	Yes	Designed primarily for testing autonomous driving planning tasks.
	Ego Simulation	Yes	Yes	Including Ego Motion Controller and Ego Update Model for closed-loop simulation.
	Agents Simulation (Agents Update Policy)	Yes	Yes	Used to simulate interactive behavior of other vehicles, ensuring comprehensive testing of obstacle avoidance algorithms.
	Metrics Evaluation	Yes	Yes	Users can combine metrics like Safety, Efficiency, Smoothness, and Task Completion according to task priorities.
	Scenario Visualization	/	Yes	Supporting user customization for adding relevant information.

Table 8 lists the current methods provided by the core components of the Simulation Engine for the dynamic obstacle avoidance test.

Table 8

Core component configuration for dynamic obstacle avoidance test.

Components	Current Methods	Explanation
Prediction Algorithms	1) Perfect prediction; 2) Constant velocity and yaw rate kinematics prediction; 3) Multimodal Trajectory Prediction model	It is recommended to use Method 1 and Method 2. The network parameters for Method 3 currently cannot be modified.
Planning Algorithms	1) Simple longitudinal planner based on Intelligent Driving Model (IDM); 2) Sampling Planner based on Predefined Maneuver Modes (SPPMM)	MineSim provides two benchmark algorithms.
Ego Simulation (Ego open-loop simulation)	Perfect track planned trajectory	/
Ego Simulation (Ego closed-loop simulation)	Ego Motion Controller: 1) LQR-based Controller; 2) iLQR-based Controller; 3) Pure Pursuit Controller Ego Update Model: 1) Kinematic Bicycle Model (KBM); 2) Kinematic Bicycle Model with Response Lag (KBM-wRL); 3) Kinematic Bicycle Model with Response Lag and Road Slope (KBM-wRLwRS)	/
Agents Simulation (non-reactive agents)	Replay policy	/
Agents Simulation (reactive agents)	1) IDM-based reactive policy; 2) Improved IDM-based reactive policy; 3) Multimodal Trajectory Prediction-based reactive policy;	/

Table 9 lists the current methods provided by the core components of the Simulation Engine for the static obstacle avoidance test.

Table 9

Core component configuration for static obstacle avoidance test.

Components	Current Methods	Explanation
Prediction Algorithms	/	/
Planning Algorithms	1) Sampling Planner based on Predefined Maneuver Modes (SPPMM); 2) A two-stage planner that combines Hybrid A* with Constrained iterative Linear Quadratic Regulator (HA*-CILQR)	MineSim provides two benchmark algorithms.
Ego Simulation (Ego open-loop simulation)	Perfect track planned trajectory	/
Ego Simulation (Ego closed-loop simulation)	Ego Motion Controller: 1) LQR-based Controller; 2) iLQR-based Controller; 3) Pure Pursuit Controller	/

(continued on next page)

Table 9 (continued)

Components	Current Methods	Explanation
	Ego Update Model: 1) Kinematic Bicycle Model (KBM); 2) Kinematic Bicycle Model with Response Lag (KBM-wRL); 3) Kinematic Bicycle Model with Response Lag and Road Slope (KBM-wRLwRS)	/
Agents Simulation (non-reactive agents)	Replay policy	/
Agents Simulation (reactive agents)	1) IDM-based reactive policy; 2) Improved IDM-based reactive policy; 3) Multimodal Trajectory Prediction-based reactive policy;	/

We recommend testing the newly proposed planning algorithms using the configurations listed in Table 10:

Table 10

Recommended test mode configurations for planning tasks.

Test Mode	Applicable Scenario	Explanation
1 Replay Test Mode (Ego closed-loop with non-reactive agents)	Dynamic	- Prediction Algorithms: Perfect Prediction; - Ego Closed-loop Simulation: LQR-based controller + KBM-wRL ego update model; - Agents Simulation: replay policy.
2 Interactive Test Mode (Ego closed-loop with reactive agents)	Dynamic	- Prediction Algorithms: Constant Velocity and Yaw Rate Kinematics Prediction; - Ego Closed-loop Simulation: LQR-based controller + KBM-wRL ego update model; - Agents Simulation: Improved IDM-based reactive policy.
3 Static Replay Test Mode	Static	Ego Closed-loop Simulation: LQR-based controller + KBM-wRL ego update model;

Data availability

We have provided the links to the data and code in the manuscript, which have been shared on GitHub.

References

- Althoff, M., Koschi, M., Manzing, S., 2017. CommonRoad: Composable benchmarks for motion planning on roads. In: In: 2017 IEEE Intelligent Vehicles Symposium (IV). <https://doi.org/10.1109/IVS.2017.7995802>.
- Baidu, 2024. Apollo: An open autonomous driving platform [WWW Document]. URL <https://github.com/ApolloAuto/apollo>.
- Bergamini, L., Ye, Y., Scheel, O., Chen, L., Hu, C., Del Pero, L., Osinski, B., Grimmett, H., Ondruska, P., 2021. SimNet: learning reactive self-driving simulations from real-world observations. In: In: IEEE International Conference on Robotics and Automation (ICRA). <https://doi.org/10.1109/ICRA48506.2021.9561666>.
- Caesar, H., Bankiti, V., Lang, A.H., Vora, S., Liong, V.E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G., Beijbom, O., 2020. nuScenes: A multimodal dataset for autonomous driving. In: In: the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). <https://doi.org/10.1109/CVPR42600.2020.01164>.
- Caesar, H., Kabzan, J., Tan, K.S., Fong, W.K., Wolff, E., Lang, A., Fletcher, L., Beijbom, O., Omari, S., 2022. NuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles. <https://doi.org/10.48550/arXiv.2106.11810>.
- Dolgov, D., Thrun, S., Montemerlo, M., Diebel, J., 2010. Path Planning for autonomous vehicles in unknown semi-structured environments. The International Journal of Robotics Research 29, 485–501. <https://doi.org/10.1177/0278364909359210>.
- Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V., 2017. CARLA: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning (CoRL).
- Ettinger, S., Cheng, S., Caine, B., Liu, C., Zhao, H., Pradhan, S., Chai, Y., Sapp, B., Qi, C. R., Zhou, Y., Yang, Z., Chouard, A., Sun, P., Ngiam, J., Vasudevan, V., McCauley, A., Shlens, J., Anguelov, D., 2021. Large scale interactive motion forecasting for autonomous driving: the Waymo Open Motion Dataset. Presented at the Proceedings of the IEEE/CVF International Conference on Computer Vision (CVPR). <https://doi.org/10.1109/ICCV48922.2021.00957>.
- Fan, H., Zhu, F., Liu, C., Zhang, L., Zhuang, L., Li, D., Zhu, W., Hu, J., Li, H., Kong, Q., 2018. Baidu apollo EM motion planner. [online]. <http://arxiv.org/abs/1807.08048>.
- Feng, L., Li, Q., Peng, Z., Tan, S., Zhou, B., 2023a. TrafficGen: learning to generate diverse and realistic traffic scenarios. In: In: IEEE International Conference on Robotics and Automation (ICRA). <https://doi.org/10.1109/ICRA48891.2023.10160296>.
- Feng, S., Sun, H., Yan, X., Zhu, H., Zou, Z., Shen, S., Liu, H.X., 2023b. Dense reinforcement learning for safety validation of autonomous vehicles. Nature 615, 620–627. <https://doi.org/10.1038/s41586-023-05732-2>.
- Gulino, C., Fu, J., Luo, W., Tucker, G., Bronstein, E., Lu, Y., Harb, J., Pan, X., Wang, Y., Chen, X., 2023. Waymax: an accelerated, data-driven simulator for large-scale autonomous driving research. Adv. Neur. In. 36, 7730–7742.
- Hendriks, F., Tideman, M., Pelders, R., Bours, R., Liu, X., 2010. Development tools for active safety systems: Prescan and VeHIL, in. In: Proceedings of 2010 IEEE International Conference on Vehicular Electronics and Safety (CVES). <https://doi.org/10.1109/ICVES.2010.5550948>.
- Houston, J., Zuidhof, G., Bergamini, L., Ye, Y., Chen, L., Jain, A., Omari, S., Iglovikov, V., Ondruska, P., 2021. One thousand and one hours: Self-driving motion prediction dataset, in: Conference on Robot Learning (CoRL).
- Jia, L., Yang, D., Ren, Y., Qian, C., Feng, Q., Sun, B., Wang, Z., 2024. A dynamic test scenario generation method for autonomous vehicles based on conditional generative adversarial imitation learning. Accid. Anal. Prev. 194, 107279. <https://doi.org/10.1016/j.aap.2023.107279>.
- Jiang, X., Zhao, X., Liu, Y., Li, Z., Hang, P., Xiong, L., Sun, J., 2024. InterHub: a naturalistic trajectory dataset with dense interaction for autonomous driving. <https://doi.org/10.48550/arXiv.2411.18302>.
- Kim, B., Yi, K., 2014. Probabilistic and holistic prediction of vehicle states using sensor fusion for application to integrated vehicle safety systems. IEEE Trans. Intell. Transport. Syst. 15, 2178–2190. <https://doi.org/10.1109/TITS.2014.2312720>.
- Levenberg, K., 1944. A method for the solution of certain non-linear problems in least squares. Q. Appl. Math. 2 (2), 164–168.
- Li, H., Chen, P., Yu, G., Zhou, B., Han, Z., Liao, Y., 2024a. Collaborative trajectory planning for autonomous mining trucks: a grouping and prioritized optimization-based approach. IEEE Trans. Veh. Technol. 73, 6283–6300. <https://doi.org/10.1109/TVT.2023.3343703>.
- Li, Q., Peng, Z. (Mark), Feng, L., Liu, Z., Duan, C., Mo, W., Zhou, B., 2023b. ScenarioNet: open-source platform for large-scale traffic scenario simulation and modeling. Adv. Neur. In. 36, 3894–3920.
- Li, L., Chen, Z., Wang, J., Zhou, B., Yu, G., Chen, X., 2024. Multimodal trajectory prediction for autonomous driving on unstructured roads using deep convolutional network. <https://doi.org/10.48550/arXiv.2409.18399>.
- Li, B., Ouyang, Y., Li, L., Zhang, Y., 2022a. Autonomous driving on curvy roads without reliance on Frenet frame: a cartesian-based trajectory planning method. IEEE Trans. Intell. Transport. Syst. 23, 15729–15741. <https://doi.org/10.1109/TITS.2022.3145389>.
- Li, B., Fan, L., Ouyang, Y., Tang, S., Wang, X., Cao, D., Wang, F.-Y., 2023a. Online competition of trajectory planning for automated parking: benchmarks, achievements, learned lessons, and future perspectives. IEEE Trans. Intell. Veh. 8, 16–21. <https://doi.org/10.1109/TIV.2022.3228963>.
- Li, Y., Li, Z., Teng, S., Zhang, Y., Zhou, Y., Zhu, Y., Cao, D., Tian, B., Ai, Y., Xuanyuan, Z., Chen, L., 2022b. AutoMine: an unmanned mine dataset. In: In: the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 21308–21317.
- Li, B., Ouyang, Y., Li, X., Cao, D., Zhang, T., Wang, Y., 2023b. Mixed-integer and conditional trajectory planning for an autonomous mining truck in loading/dumping scenarios: a Global Optimization Approach. IEEE Trans. Intell. Veh. 8, 1512–1522. <https://doi.org/10.1109/TIV.2022.3214777>.
- Li, Q., Peng, Z., Feng, L., Zhang, Q., Xue, Z., Zhou, B., 2023d. MetaDrive: composing diverse driving scenarios for generalizable reinforcement learning. IEEE Trans. Pattern Anal. Mach. Intell. 45, 3461–3475. <https://doi.org/10.1109/TPAMI.2022.3190471>.
- Li, B., Wang, Y., Ma, S., Bian, X., Li, H., Zhang, T., Li, X., Zhang, Y., 2023c. Adaptive pure pursuit: a real-time path planner using tracking controllers to plan safe and kinematically feasible paths. IEEE Trans. Intell. Veh. 8, 4155–4168. <https://doi.org/10.1109/TIV.2023.3296435>.

- Lopez, P.A., Wiessner, E., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flotterod, Y.-P., Hilbrich, R., Lucken, L., Rummel, J., Wagner, P., 2018. Microscopic traffic simulation using SUMO. In: In: International Conference on Intelligent Transportation Systems (ITSC). <https://doi.org/10.1109/ITSC.2018.8569938>.
- Meng, S., Tian, B., Zhang, X., Qi, S., Zhang, C., Zhang, Q., 2024. OpenMines: a light and comprehensive mining simulation environment for truck dispatching, in: 2024 IEEE Intelligent Vehicle Symposium (IV). <https://doi.org/10.1109/IV55156.2024.10588764>.
- Mo, X., Liu, H., Huang, Z., Fang, J., Xue, J., Lv, C., 2024. Scalable traffic simulation for autonomous driving via multi-agent goal assignment and autoregressive goal-directed planning. In: In: 2024 IEEE Intelligent Vehicles Symposium (IV). <https://doi.org/10.1109/IV55156.2024.10588681>.
- Montali, N., Lambert, J., Mouglin, P., Kuefler, A., Rhinehart, N., Li, M., Gulino, C., Emrich, T., Yang, Z., Whiteson, S., 2024. The waymo open sim agents challenge. *Adv. Neur. In.* 36, 59151–59171.
- Rajamani, R., 2012. *Vehicle dynamics and control*, Second, Edition. ed. Springer Science & Business Media, New York, USA.
- Rong, G., Shin, B.H., Tabatabaee, H., Lu, Q., Lemke, S., Mozeiko, M., Boise, E., Uhm, G., Gerow, M., Mehta, S., Agafonov, E., Kim, T.H., Sterner, E., Ushiroda, K., Reyes, M., Zelenkovsky, D., Kim, S., 2020. LGSVL Simulator: a high-fidelity simulator for autonomous driving. In: In: 2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC). <https://doi.org/10.1109/ITSC45102.2020.9294422>.
- Shah, S., Dey, D., Lovett, C., Kapoor, A., 2018. AirSim: High-fidelity visual and physical simulation for autonomous vehicles. In: Hutter, M., Siegwart, R. (Eds.), *Field and Service Robotics*, Springer Proceedings in Advanced Robotics. Springer International Publishing, Cham, pp. 621–635. https://doi.org/10.1007/978-3-319-67361-5_40.
- Sun, S., Liu, Z., Yin, H., Ang, M.H., 2022. FISS: a trajectory planning framework using fast iterative search and sampling strategy for autonomous driving. *IEEE Robot. Autom. Lett.* 7, 9985–9992. <https://doi.org/10.1109/LRA.2022.3191940>.
- Sun, S., Chen, J., Sun, J., Yuan, C., Li, Y., Zhang, T., Ang, M.H., 2023. In: FISS+: Efficient and Focused Trajectory Generation and Refinement Using Fast Iterative Search and Sampling Strategy, in: <https://doi.org/10.1109/IROS55552.2023.10341498>.
- Tassa, Y., Mansard, N., Todorov, E., 2014. In: Control-Limited Differential Dynamic Programming, in: <https://doi.org/10.1109/ICRA.2014.6907001>.
- Treiber, M., Hennecke, A., Helbing, D., 2000. Congested traffic states in empirical observations and microscopic simulations. *PhysRevE* 62, 1805–1824. <https://doi.org/10.1103/PhysRevE.62.1805>.
- Usami, R., Kobashi, Y., Onuma, T., Maekawa, T., 2020. Two-lane path planning of autonomous vehicles in 2.5D environments. *IEEE Trans. Intell. Veh.* 5, 281–293. <https://doi.org/10.1109/ITV.2019.2955904>.
- Wang, Y., Zhao, T., Yi, F., 2023. Multiverse transformer: 1st place solution for waymo open sim agents challenge 2023. <https://doi.org/10.48550/arXiv.2306.11868>.
- Werling, M., Ziegler, J., Kammel, S., Thrun, S., 2010. Optimal trajectory generation for dynamic street scenarios in a Frenet frame. In: *IEEE International Conference on Robotics and Automation (ICRA)*.
- Xu, Z., Jiang, T., Zheng, N., 2022. Developing and analyzing eco-driving strategies for on-road emission reduction in urban transport systems - a VR-enabled digital-twin approach. *Chemosphere* 305, 135372. <https://doi.org/10.1016/j.chemosphere.2022.135372>.
- Xu, S., Peng, H., Tang, Y., 2021. Preview path tracking control with delay compensation for autonomous vehicles. *IEEE Trans. Intell. Transport. Syst.* 22, 2979–2989. <https://doi.org/10.1109/TITS.2020.2978417>.
- Xu, Z., Zheng, N., Logan, D.B., Vu, H.L., 2023. Assessing bicycle-vehicle conflicts at urban intersections utilizing a VR integrated simulation approach. *Accid. Anal. Prev.* 191, 107194. <https://doi.org/10.1016/j.aap.2023.107194>.
- Xu, Z., Zheng, Z., Xiao, D., Tu, R., Ma, W., Zheng, N., 2024. Assessing the impact of passenger compliance behavior in CAVs on environmental benefits. *Transp. Res. Part D: Transp. Environ.* 133, 104278. <https://doi.org/10.1016/j.trd.2024.104278>.
- Zhou, X., Wang, Z., Ye, H., Xu, C., Gao, F., 2021. EGO-Planner: an ESDF-Free gradient-based local planner for quadrotors. *IEEE Robot. Autom. Lett.* 6, 478–485. <https://doi.org/10.1109/LRA.2020.3047728>.